

ElderAssist



Mubarak Albraik 443170244

Mohammed Alsumaykhi 443100933

Abdullah Benjbr 443170266

Supervised By

Dr. Mohamed Maddeh

Submitted for the partial fulfillment of Bachelor of Computer and Information

Sciences in Applied computer science degree

College of Computer and Information Science, AlMuzahmiyah Campus

KING SAUD UNIVERSITY

2025-2026

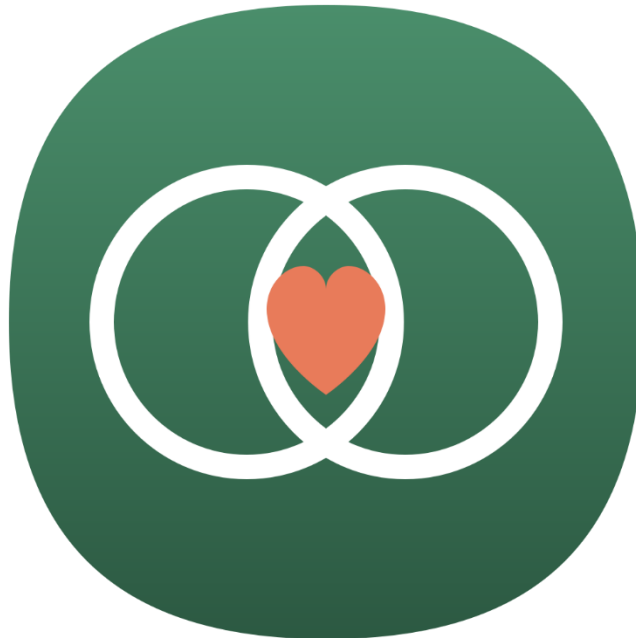
ABSTRACT

ElderAssist is a user-friendly mobile application designed to enhance the independence, safety, and social connectivity of elderly users. The app offers essential features such as emergency alerts, medication reminders, health monitoring, and easy communication channels with family members and caregivers. Its intuitive interface ensures accessibility for users with varying levels of tech familiarity, incorporating large buttons, simple navigation, and voice instructions. By providing a comprehensive platform for health management and social interaction, ElderAssist aims to improve the quality of life for seniors, foster a sense of security, and facilitate closer connections with loved ones and support networks.



ElderAssist

SIMPLE SUPPORT FOR SENIORS



DECLARATION

We hereby declare that our dissertation is entirely our work and genuine / original. We understand that in case of discovery of any PLAGIARISM at any stage, our group will be assigned an F (FAIL) grade and it may result in withdrawal of our Bachelor's degree.

Group members:

Name

Signature

Mubarak Albraik

Mohammed Alsumaykhi

Abdullah Benjbr

Table of Contents

<i>Chapter 1 : INTRODUCTION</i>	10
1.1 Problem statement	10
1.2 Objectives and motivations	10
1.3 Purposed system	11
1.4 Development methodology	11
1.4.1 V-shaped	11
1.4.2 Gantt Chart	12
<i>Chapter 2 : BACKGROUND</i>	13
2.1 Comparison to competitors	13
<i>Chapter 3 : SYSTEM ANALYSIS</i>	14
3.1 Requirement Engineering	14
3.1.1 Functional requirements	14
3.1.2 Nonfunctional requirements	15
3.1.3 Technical Requirements	15
3.2 Use Case	16
3.2.1 Use Case Diagram	16
UC-01 Register a User:	17
UC-02 Login:	18
UC-03A Add Medication:	19
UC-03B Edit Medication:	20
UC-03C Delete Medication:	21
UC-04 – Trigger SOS:	22
UC-05 – Smart Supplement:	23
UC-06 – Manage Appointment:	25
UC-07 – Call for Help:	26
<i>Chapter 4 : SYSTEM DESIGN</i>	27
4.1 Class Diagram	27
4.2 Sequence Diagrams	28

4.2.1 Sequence Diagram: Register a User	28
4.2.2 Sequence Diagram: login to a User	29
4.2.3 Sequence Diagram: Manage Medication	30
4.2.4 Sequence Diagram: SOS trigger	31
4.2.5 Sequence Diagram: Smart Supplement Assistant.....	32
4.2.6 Sequence Diagram: Manage Appointment	33
4.2.7 Sequence Diagram: Call for Help / Rating	33
4.3 ER Diagram	35
Chapter 5 Implementation	36
5.1 The Frontend	36
5.1.1 Authentication	37
5.1.2 The Senior	43
5.1.3 The Caregiver	47
5.1.4 Shared Screens	49
5.2 The Backend (Firebase Cloud Services)	51
5.2.1 Authentication	51
5.2.2 The Cloud Firestore Data Model	52
5.2.3 Firestore Security Rules	56
5.2.4 Cloud Functions	56
5.2.5 Project Configuration and Deployment.....	57
5.3 The AI Integration	58
Chapter 6 Testing	61
6.1 Introduction	61
6.1.1 Unit Testing.....	62
6.1.2 Integration Testing	62
6.2 Unit Test	62
6.2.1 Test Case 1: Sign In (Phone + OTP).....	62
6.2.2 Test Case 2: PIN Unlock	63
6.2.3 Test Case 3: Daily Check-in	63
6.2.4 Test Case 4: Add Medication.....	64
6.2.5 Test Case 5: Health Assistant Chat.....	65

6.3	Integration Test	66
6.3.1	Senior and Caregiver Linking.....	66
6.3.2	Care Chat and Medication Proposal.....	67
6.3.3	Notifications and Reminders.....	67
6.3.4	Medication Adherence Pipeline	68
6.4	User Acceptance Test (UAT)	68
6.4.1	Senior’s Module Test.....	69
6.4.2	Caregiver’s Module Test.....	69
6.4.3	General Usability and Trust.....	70
6.5	Summary.....	70
Chapter 7 CONCLUSION		71
7.1	Conclusion	71
7.2	Future Work	71
7.3	Final Thoughts	72
REFERENCES		72

Table of Figures

Figure 1-1 SHAPED DEVELOPMENT PROCESS-----	12
Figure 1-2 Gantt Chart -----	12
Figure 4-1 Class Diagram-----	27
Figure 4-2 Register a User Sequence Diagram -----	28
Figure 4-3 Login to User Sequence Diagram-----	29
Figure 4-4 Manage Medications Sequence Diagram -----	30
Figure 4-5 Trigger SOS Sequence Diagram-----	31
Figure 4-6 Smart Supplement Assistant Sequence Diagram -----	32
Figure 4-7 Manage Appointment Sequence Diagram -----	33
Figure 4-8 Call for Help Sequence Diagram -----	34
Figure 4-9 ER Diagram -----	35
Figure 5-1 Splash screen -----	37
Figure 5-2 Phone input page-----	38
Figure 5-3 OTP verification page -----	38
Figure 5-4 Email input page-----	39
Figure 5-5 Password sign-in page-----	40
Figure 5-6 Create password page-----	40
Figure 5-7 Verify email page -----	41
Figure 5-8 Role selection page -----	41
Figure 5-9 Set PIN page-----	42
Figure 5-10 PIN unlock page-----	42
Figure 5-11 Forgot PIN re-authentication page-----	43
Figure 5-12 Reset PIN page -----	43
Figure 5-13 Senior profile setup page -----	44
Figure 5-14 Senior home dashboard -----	44
Figure 5-15 Medication list page -----	45
Figure 5-16 Add medication page-----	46
Figure 5-17 Link caregiver page -----	46
Figure 5-18 AI health assistant chat page -----	47
Figure 5-19 Caregiver profile setup page -----	47
Figure 5-20 Caregiver home page -----	48
Figure 5-21 Caregiver linking code page-----	48
Figure 5-22 Caregiver dashboard-----	49
Figure 5-23 Senior detail page-----	49
Figure 5-24 My caregiver page-----	50
Figure 5-25 Care chat page -----	50

Figure 5-26 Medication proposal form-----	51
Figure 5-27 Settings page -----	51
Figure 5-28 Firebase Authentication — Users tab -----	52
Figure 5-29 Authentication — Sign-in method -----	52
Figure 5-30 Firestore — collections overview -----	53
Figure 5-31 Firestore — a sample user document-----	54
Figure 5-32 Firestore — medications sub-collection -----	54
Figure 5-33 Firestore — doseLogs sub-collection -----	54
Figure 5-34 Firestore — checkIns sub-collection-----	55
Figure 5-35 Firestore — chatHistory sub-collection -----	55
Figure 5-36 Firestore — linkCodes collection -----	55
Figure 5-37 Firestore — Security Rules tab -----	56
Figure 5-38 Cloud Functions — Functions tab -----	57
Figure 5-39 Firebase Project settings — General tab -----	57
Figure 5-40 firebase.json -----	58
Figure 5-41 The ai_rules.txt system prompt file -----	59
Figure 5-42 Vertex AI — Model Garden / Studio -----	59
Figure 5-43 Cloud Functions — Logs -----	60
Figure 5-44 Firestore — populated chatHistory after a session-----	60

List of Tables

Table 2.1 Comparison to Competitors	13
Table 3.1 Use Case UC-01: Register a User	17
Table 3.2 Use Case UC-02: Login.....	18
Table 3.3 Use Case UC-03A: Add Medication	19
Table 3.4 Use Case UC-03B: Edit Medication.....	20
Table 3.5 Use Case UC-03C: Delete Medication	21
Table 3.6 Use Case UC-04: Trigger SOS.....	22
Table 3.7 Use Case UC-05: Smart Supplement.....	23
Table 3.8 Use Case UC-06: Manage Appointment.....	25
Table 3.9 Use Case UC-07: Call for Help	26
Table 6.1 Test Case: Sign In	63
Table 6.2 Test Case: PIN Unlock	63
Table 6.3 Test Case: Daily Check-in	64
Table 6.4 Test Case: Add Medication.....	65
Table 6.5 Test Case: Health Assistant Chat	65
Table 6.6 Integration Test: Linking	66
Table 6.7 Integration Test: Care Chat.....	67
Table 6.8 Integration Test: Notifications	68
Table 6.9 Integration Test: Adherence	68
Table 6.10 Senior's Module Test	69
Table 6.11 Caregiver's Module Test	70
Table 6.12 General Usability Test	70

Chapter 1 : INTRODUCTION

1.1 Problem statement

The aging population often faces challenges related to health management, safety, and social isolation, which can impact on their overall well-being and independence. Existing solutions are often complex or not tailored to the specific needs of elderly users, making it difficult for them to access essential services and stay connected with family and caregivers. There is a need for a simple, accessible, and reliable mobile application that supports elderly individuals in managing their health, ensuring their safety, and maintaining social interactions, thereby enhancing their quality of life and promoting independent living.

1.2 Objectives and motivations

1. Enhance Independence: Provide elderly users with tools that enable them to manage daily activities (medication, health, safety) without constant assistance.
2. Improve Safety: Integrate emergency alerts and health monitoring features to ensure quick responses in critical situations.
3. Promote Social Connectivity: Offer easy-to-use communication channels to reduce loneliness and strengthen ties with family, caregivers, and support networks.
4. Address Aging Population Needs: With increasing elderly populations worldwide, there is a growing demand for accessible digital health and support solutions.
5. Reduce Caregiver Burden: Simplify monitoring and communication for caregivers, easing their workload while ensuring elders remain cared for.
6. Bridge Technology Gaps: Create an intuitive, senior-friendly interface that overcomes common barriers older adults face with complex digital tools.

1.3 Purposed system

The proposed system, *ElderAssist*, is a mobile application specifically designed to support the elderly in managing their health, safety, and social well-being. The system integrates multiple features into a single platform to ensure ease of use and accessibility.

The app provides emergency alert services that allow seniors to instantly notify family members, caregivers, or medical services in critical situations. It includes medication reminders with notifications and voice alerts to help users take medicines on time. Additionally, the system supports health monitoring, where users can track vital information such as blood pressure, heart rate, and daily activity, either manually or through wearable device integration.

To promote social interaction, *ElderAssist* offers simplified communication tools such as video calls, text messaging with large fonts, and voice-assisted commands, enabling users to stay connected with family and caregivers easily. The interface is designed with large icons, simple navigation, and voice guidance, making it highly accessible even for those with limited technical experience.

By combining health management, safety monitoring, and social connectivity in a single, easy-to-use mobile platform, the proposed system aims to improve the independence, quality of life, and emotional well-being of elderly users, while also offering reassurance and support for their families and caregivers.

1.4 Development methodology

Our chosen methodology is V-shaped Development methodology, because this method assures that mistakes and errors are detected and dealt with in an early phase of development. Not only that, but this method helps the developers to have clear and concise idea about what the project is about and how to be familiar with the mistakes encountered.

1.4.1 V-shaped

The V-shaped model is a structured software development approach that emphasizes verification and validation at every stage. It consists of distinct phases where each phase on the left side (verification) corresponds to a matching phase on the right side (validation). The process begins with Requirement Gathering, followed by System Analysis, Software Design, and Module Design, where each phase ensures that the project's objectives are clearly defined and planned. Coding forms the base of the V-shape, where the actual implementation takes place. On the validation side, Unit Testing, Integration Testing, System Testing, and Acceptance Testing are carried out to

validate the code against the initial requirements and ensure all components function together as

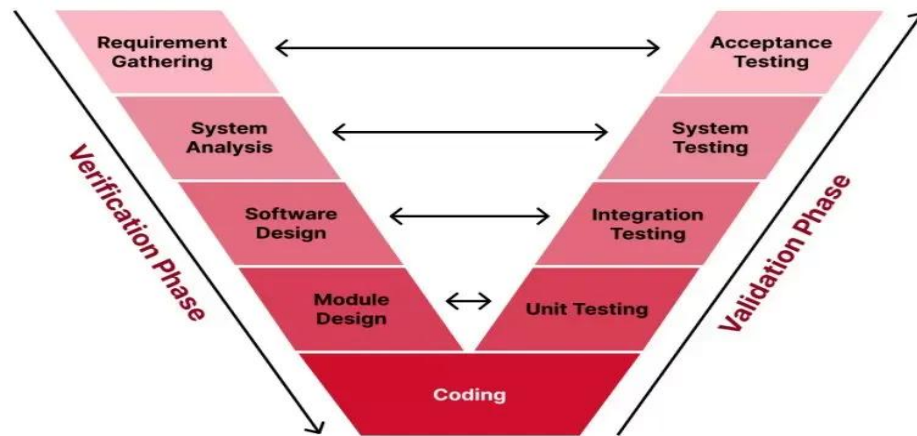


Figure 1-1 SHAPED DEVELOPMENT PROCESS

1.4.2 Gantt Chart

The Gantt chart provides a timeline for the *ElderAssist* project, illustrating the sequential tasks and the duration required to complete each phase of development. The process begins with Requirement Gathering and progresses through key phases like System Analysis, Software Design, and Module Design, followed by the actual Coding phase. Post-coding, testing phases including Unit Testing, Integration Testing, System Testing, and Acceptance Testing are carried out to ensure quality. The chart shows the dependencies between tasks and highlights the critical path, ensuring that each stage is completed in a timely manner to meet the project's deadline.

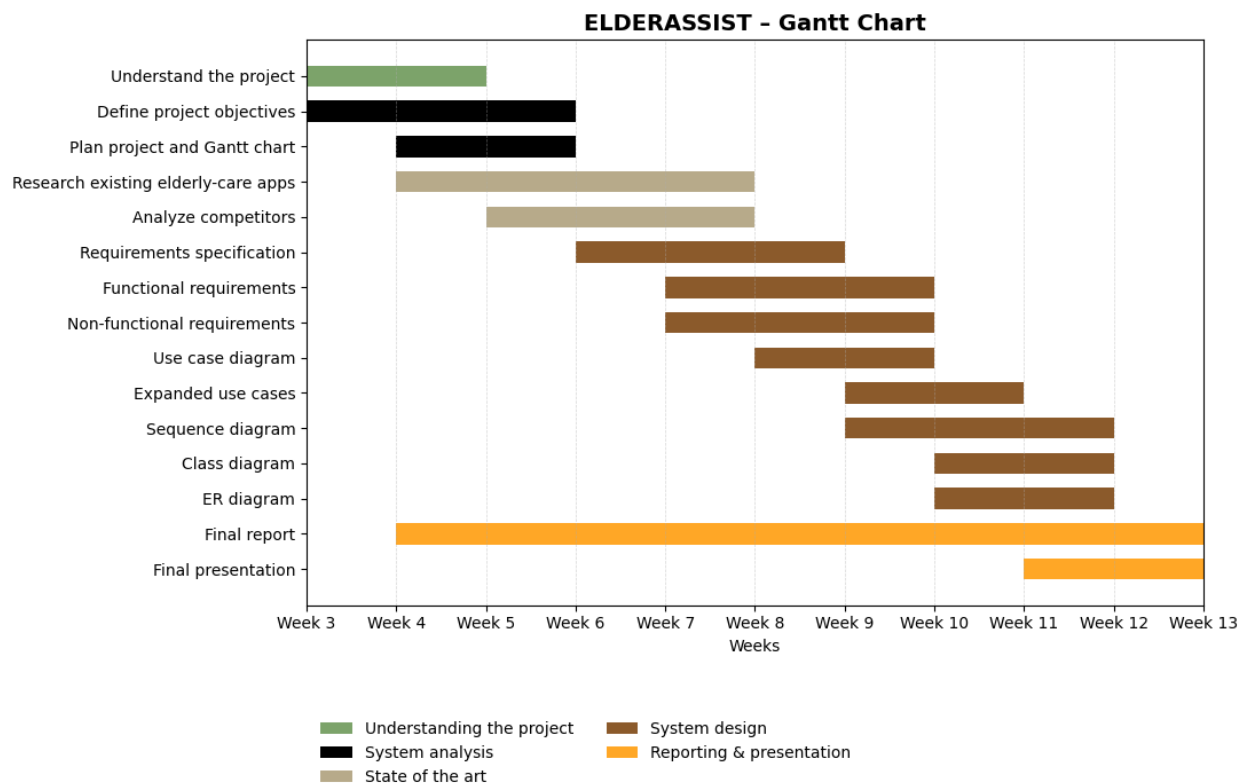


Figure 1-2 Gantt Chart

Chapter 2 : BACKGROUND

2.1 Comparison to competitors

There are many applications that support older adults and their families. However, most of them focus on only one part of the problem (medication reminders, communication, or information). ElderAssist aims to combine emergency help, daily medication support, simple communication, and guidance (such as the Smart Supplement Assistant) in one easy app.

Table 2.1 Comparison to Competitors¹

App	Purpose	Main Features	Limitations
CareZone 	Manages medicines and doctor visits	• Medication reminders • Appointment calendar • Stores medical files • Family access	• No emergency button • Hard for some seniors • No smart device support
Pill Reminder – Meds Alarm 	Medication reminders only	• Simple alarms • Large text • Works offline • Multiple medicines	• No health records • No family tracking • Limited features
Be My Eyes 	Visual help via video call	• Free • Global volunteers • Daily task help • Many languages	• Needs internet • Privacy concerns • No health support
Medisafe 	Medication management	• Pill reminders • Drug interaction alerts • Family alerts • Multi-device support	• Ads in free version • Too many alerts • Complex for some users
CaringBridge 	Health updates for families	• Share health updates • Emotional support • Free	• No reminders • Needs internet • Family-focused
Oscar Senior 	Communication and reminders	• Large buttons • Calls, texts, photos • Emergency button	• Paid app • Limited customization

Chapter 3 : SYSTEM ANALYSIS

This chapter presents the analysis of the requirements and technical specifications of *ElderAssist*. It outlines the functional and non-functional requirements necessary to implement the system. The analysis also includes user requirements such as SOS alerts, medication reminders, and contact management. The chapter further describes the technical architecture, including the mobile client, local storage, notifications, and system integrations. This chapter presents the analysis of the requirements and technical specifications of *ElderAssist*. It outlines the functional and non-functional requirements necessary to implement the system.

3.1 Requirement Engineering

3.1.1 Functional requirements

1. **User Registration & Login:** First run creates a profile (name , primary phone, primary emergency contact, 4–6-digit PIN). Subsequent access unlocks the app with **PIN**.
2. **Start-up Config:** First-run flow to set text size, add one emergency contact, and optionally add one medication.
3. **“I’m Okay” Button:** A single daily prompt with one button: “I’m okay today”. If not pressed by a set time, notify a family member.
4. **Medication Manager:** Add/edit/delete medications with name + time; list shown in chronological order; mark “Taken” with one tap.
5. **Call for help:** the possibility of helping the Senior User remotely through video.
6. **Smart AI Health Assistant (LLM-Based):**
A conversational AI interface powered by Gemini 2.5 Flash via Vertex AI.
The assistant allows Seniors to describe symptoms in natural language.
The system analyzes user input and provides general wellness guidance and supplement suggestions.
7. **Reminder Service:** The system shall provide a Reminder Service that delivers notifications to the Senior User and/or Caregiver for scheduled medications, upcoming appointments, and other time-based tasks defined in the app.
8. **Rate Caregiver:** allows the user to rate the service provided by Care giver
9. **Manage appointment:** Book an appointment for the selected clinic and view the nearest available times and locations for the closest hospitals.
10. **Caregiver Consultation:** Allows the Caregiver to start voice/video check-ins with the Senior User to discuss health concerns.
11. **Report a Problem:** allows the user to report all problems related to the Care Giver
12. **Answer for query:** It allows the assistant to respond to the call for help for the elderly.

3.1.2 Nonfunctional requirements

1. **Usability:** Core tasks (SOS, add med, call contact) in **≤3 taps**; clear labels, The app must have an intuitive, user-friendly interface, especially for older users.
2. **Performance:** Cold start **≤5.0s** on low-end Android; tap response **≤100 ms**.
3. **Reliability:** **≥95%** timely local notifications when OS permissions are granted.
4. **Accessibility:** Tap targets **≥48×48dp**; supports system font scaling and screen readers; high-contrast theme.
5. **Privacy & Security:** User authentication and core data (profiles, medications, appointments, SOS events, chat logs) shall be stored securely in Firebase Firestore. Sensitive AI prompts shall be processed through secure backend cloud functions..
6. **Platform Compatibility:** Flutter app optimized for **Android** devices.
7. **Accuracy & Robustness:** Voice recognition should correctly detect supported commands in quiet environments with **≥85%** accuracy for clearly spoken phrases.

3.1.3 Technical Requirements

1. Flutter SDK; Android toolchain
2. Local DB: SQLite

3.2 Use Case

3.2.1 Use Case Diagram

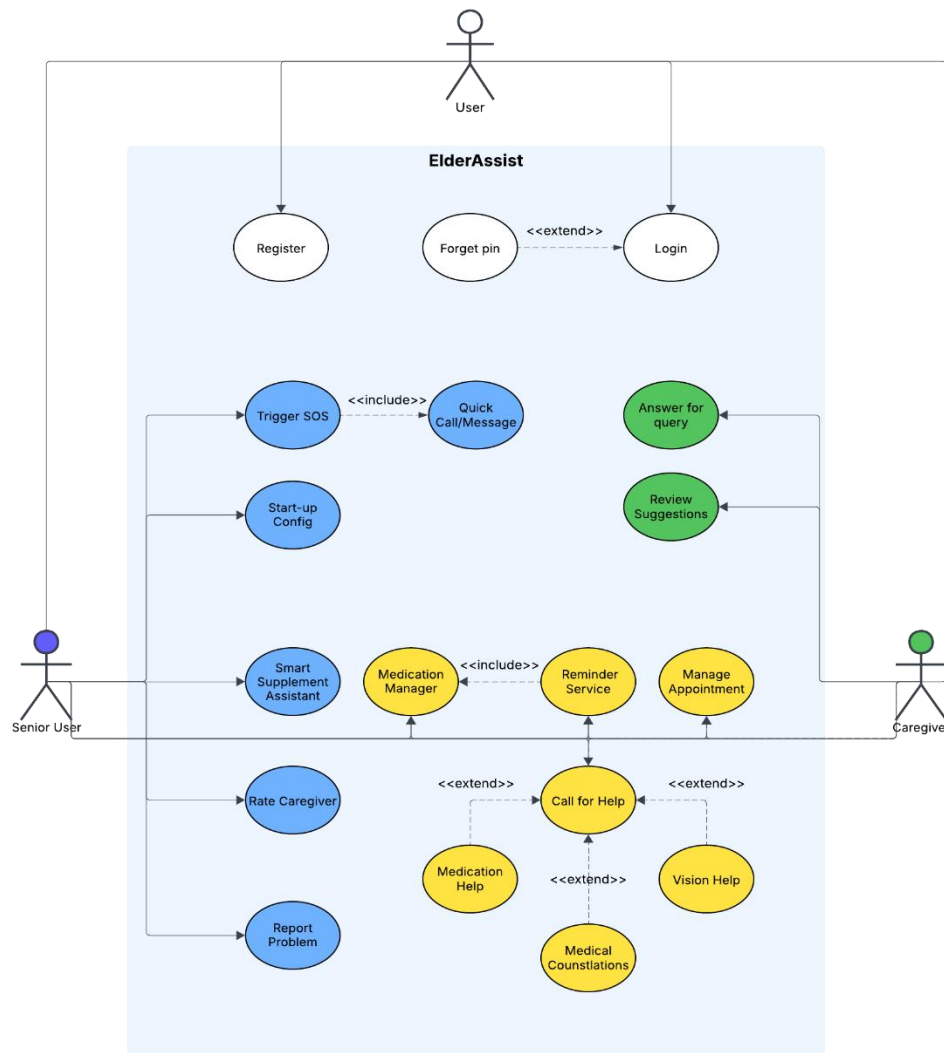
The use case diagram for **ElderAssist** represents two main actors: **Senior User**, **Caregiver**, Each role interacts with the system to perform specific assistance and management tasks.

1. **Senior**

The primary user of the application. The Senior User can **book appointments**, **trigger SOS alerts** (which include a quick call or message to contacts), and **use voice commands** to interact with the system. They can also **recover a forgotten PIN**, **manage favorites and personal settings**, and **manage medications** or **act on reminders** related to their daily schedule and health needs.

2. **Caregiver:**

The Caregiver supports and monitors the Senior User. They can **monitor medication adherence**, **manage emergency contacts**, and coordinate **medical consultations**. In addition, they can **view and manage appointments**, **manage medications**, and **act on reminders** to ensure that the Senior User follows their treatment and activity plans.



UC-01 Register a User:

This use case describes how a new Senior User registers in **ElderAssist** for the first time. The user creates a local profile by providing basic information and setting a **4–6 digit PIN** that will be used later to unlock the application.

Table 3.1 Use Case UC-01: Register a User2

Actor	System
1. The use case begins when the Senior User opens ElderAssist and selects “Register” from the start screen. 3. The Senior User enters the required information and chooses a 4–6 digit PIN. 4. The Senior User confirms the entered PIN by re-typing it. 5. The Senior User taps “Submit” / “Create Profile”.	2. The system displays the registration form requesting: • Name • Primary phone number • Primary emergency contact name and phone number • Desired 4–6-digit PIN • PIN confirmation 6. The system validates the input (all mandatory fields filled, PIN format is correct, both PIN entries match). 7. The system displays a confirmation message such as “Registration successful”. 8. The system redirects the user to the login screen or offers a “Continue to app” option. 9. The use case ends successfully.

Preconditions

1. ElderAssist application is installed and running on the device.
2. The user is **not yet registered** in the system.

Postconditions:

1. A new Senior User profile is created and stored locally.

UC-02 Login:

This use case describes how a **registered Senior User** unlocks and accesses the ElderAssist application using their **4–6 digit PIN**.

Table 3.2 Use Case UC-02: Login3

Actor	System
1. The use case begins when the Senior User opens ElderAssist and reaches the login screen. 3. The Senior User enters the PIN. 4. The Senior User confirms by tapping “Login” / “Unlock”.	2. The system displays a prompt asking the user to enter their 4–6 digit PIN. 5. The system validates the entered PIN against the stored PIN for that profile. 6. If the PIN is correct, the system loads the Senior User’s profile and displays the main dashboard/home screen. 7. The system loads the Senior User’s profile and displays the main dashboard/home screen. 8. The use case ends successfully.

Preconditions

1. ElderAssist application is installed and running on the device.
2. A valid Senior User profile already exists (user is registered).
3. The application is currently locked or at the login screen.

Postconditions

1. The PIN is verified.
2. The Senior User is authenticated and gains access to the main dashboard/home screen.

UC-03A Add Medication:

This use case describes how the Senior User or Caregiver adds, edits, or deletes medication entries and schedules in ElderAssist. These entries are later used to generate reminders

Table 3.3 Use Case UC-03A: Add Medication⁴

Actor	System
1. The actor opens ElderAssist and selects “Medications” from the main menu. 3. The actor chooses one of the options: Add 5. The actor fills in the details and taps “Save”..	2. The system displays the current medication list and schedule. 4. The system displays a form asking for: • Medication name • Dosage (e.g., 1 tablet, 5 ml) • Frequency (times per day or specific times) • Start date and (optional) end date • Additional notes or instructions 6. The system validates the input and creates a new medication record. 7. The system generates corresponding reminder events. 8. The updated list is shown; the use case continues or ends. 9. The use case ends successfully.

Preconditions

1. Senior User is logged in.
2. The user has access rights to modify medications (Senior User or assigned Caregiver).

Postconditions:

1. Medication list and schedules are updated in the system.
2. Future reminders are generated/updated accordingly.

UC-03B Edit Medication:

This use case describes how the Senior User or Caregiver adds, edits, or deletes medication entries and schedules in ElderAssist. These entries are later used to generate reminders

Table 3.4 Use Case UC-03B: Edit Medication

Actor	System
1. The actor opens ElderAssist and selects “Medications” from the main menu. 3. The actor chooses one of the options: Edit. 5. The actor updates fields as needed and taps “Save”.	2. The system displays the current medication list and schedule. 4. The system displays the current details for that medication. 6. The system validates changes, updates the record, and adjusts future reminders. 7. The updated list is shown. 8. The use case ends successfully.

Preconditions

1. Senior User is logged in.
2. The user has access rights to modify medications (Senior User or assigned Caregiver).
3. A Medication Entry was added before.

Postconditions:

1. Medication list and schedules are updated in the system.
2. Future reminders are generated/updated accordingly.

UC-03C Delete Medication:

This use case describes how the Senior User or Caregiver adds, edits, or deletes medication entries and schedules in ElderAssist. These entries are later used to generate reminders

Table 3.5 Use Case UC-03C: Delete Medication6

Actor	System
1. The actor opens ElderAssist and selects “Medications” from the main menu. 3. The actor selects an existing medication and chooses “Delete”.	2. The system displays the current medication list and schedule. 4. The system asks for confirmation. 5. On confirmation, the system removes the medication and cancels related future reminders. 6. The updated list is shown and the use case ends.

Preconditions

1. Senior User is logged in.
2. The user has access rights to modify medications (Senior User or assigned Caregiver).
3. A Medication Entry was added before.

Postconditions:

1. Medication list and schedules are updated in the system.
2. Future reminders are generated/updated accordingly.

UC-04 – Trigger SOS:

This use case describes how the Senior User initiates an SOS alert from ElderAssist. The system sends an emergency notification and optionally performs a **Quick Call/Message** to configured contacts.

Table 3.6 Use Case UC-04: Trigger SOS7

Actor	System
1. The use case begins when the Senior User activates the SOS button 3. If the Senior User does not cancel, the system confirms the SOS.	2. The system displays a confirmation dialog (e.g., “Send SOS?” with a short countdown). 4. The system creates an SOS event with timestamp and basic user info. 5. The system executes Quick Call/Message to the predefined emergency contact: a. Places a phone call or b. Sends an SMS / in-app notification containing “SOS” and optional location. 6. The system displays a screen indicating that help has been notified, showing contact status (e.g., “message sent”). 7. The use case ends.

Preconditions

1. Senior User is logged in.
2. Emergency contacts are already configured in the profile.
3. The device has network access and (optionally) phone/SMS capability.

Postconditions:

1. An SOS event is recorded.
2. Emergency contacts and/or caregivers receive a notification and/or call/message.

UC-05 – Smart Supplement:

This use case describes how the Senior User uses the **Smart Supplement Assistant** in **ElderAssist** to answer a short symptom survey and receive non-diagnostic supplement suggestions with a clear disclaimer.

Table 3.7 Use Case UC-05: Smart Supplement8

Actor	System
1. the user selects “Smart Supplement Assistant” from the menu. 3. The user Taps “Start survey”.. 5. The User Selects the option(s) that best describe current symptoms and taps “Next”.	2. The System Displays a welcome screen explaining that the feature is not medical advice and that the user should always consult a doctor before taking any supplement. 4. The System Shows the first question about general health (e.g., energy level, sleep, mood, joint comfort, memory, appetite) with simple multiple-choice answers. 6. The System Stores the answer and displays the next symptom-related question. 7. Steps 5–6 repeat until all questions are answered or the Senior User taps “Finish”.

10. Senior User: Reviews the suggestions and, if desired, taps “Save suggestion”.	8. The System Processes all responses using a predefined rules table configured by healthcare professionals (e.g., mapping symptoms to categories like “general wellness”, “joint support”, “memory support”). 9. The System Displays the suggested supplement categories with a short explanation for each (e.g., “may help support joint comfort”) and a prominent “This is not medical advice” disclaimer. 11. System: Prepares a simple summary view that can be shown on-screen or shared (according to app design) so the Caregiver or Healthcare Provider can review it. The use case ends.
--	--

Preconditions

1. Senior User is logged in.

Postconditions:

1. The user’s symptom responses are stored for this survey session
2. One or more **non-diagnostic supplement suggestions** are generated and optionally saved to the profile and/or shared for review with a caregiver or doctor.

UC-06 – Manage Appointment:

The user selects a clinic or hospital in ElderAssist, views the nearest available appointment times and locations, and confirms a booking. The system uses the user’s location (if allowed) to show the closest hospitals and their available time slots, then saves the chosen appointment and shows confirmation.

Table 3.8 Use Case UC-06: Manage Appointment⁹

Actor	System
1. The user opens ElderAssist, and select “Appointment Manager” from the main menu 3. The user selects a clinic/hospital or allows the app to show nearest hospitals based on location. 5. The user taps on a clinic/hospital to view available appointment times. 7. The user selects a preferred date and time slot. 8. The user taps “Confirm Appointment”.	2. The system displays options to search by clinic/hospital or use current location to find nearby hospitals. 4. The system displays a list of nearby clinics/hospitals with their names, locations, and distance. 6. The system retrieves and displays the nearest available dates and time slots for the selected clinic or hospital. 9. The system saves the appointment and displays a confirmation screen with the clinic name, location map or address, date, and time. 10. The system may optionally add a reminder entry to the user’s reminders/notifications. 11. The use case ends.

Preconditions

1. The user is logged into ElderAssist.
2. The device has an active internet connection.
3. (Optional) Location services are enabled if the user wants to see the nearest hospitals automatically.

Postconditions:

1. An appointment is booked and stored in the system; details (date, time, location) are shown to the user.
2. **UC-07 – Call for Help:**

The Senior User opens ElderAssist and starts a **Call for Help**. The app lets the user choose between three types of assistance: **vision help**, **medication help**, or **medical consultation**. Based on the selected option, the system connects the Senior User with an available Caregiver (or healthcare professional) through a remote video call so the helper can guide the Senior User in real time.

Table 3.9 Use Case UC-07: Call for Help10

Actor	System
1. The Senior User opens ElderAssist. 2. The Senior User selects “Call for Help” from the home screen. 4. The Senior User selects one of the three options. 7. The helper provides remote assistance according to the chosen option (e.g., reading labels for vision help, explaining medicine schedule, or answering medical questions).	3. The system displays three options: • Vision help • Medication help • Medical consultation 5. The system searches for an available Caregiver 6. The system starts a video call between the Senior User and the Caregiver . 8. The call ends when either side hangs up, and the system returns to the home screen or a summary screen.

Preconditions

1. Senior User is logged in.

A video call is established with a Caregiver and the selected help session starts.

Chapter 4 : SYSTEM DESIGN

This chapter will have diagrams demonstrating how the application will behave, this chapter will contain Class diagrams. Sequence diagrams.

4.1 Class Diagram

The class diagram represents a support system for senior users and caregivers. User serves as the base class, extended by SeniorUser and Caregiver for specialized roles. Senior users can trigger SOS alerts, request help, start calls, manage appointments, track medications, and receive reminders. Caregivers respond to queries and can be rated by users. The system also supports submitting problem reports and handling emergency events through Call, SOSAlert, and CallForHelp relationships. Overall, the model integrates communication, health management, scheduling, and feedback into one cohesive platform.

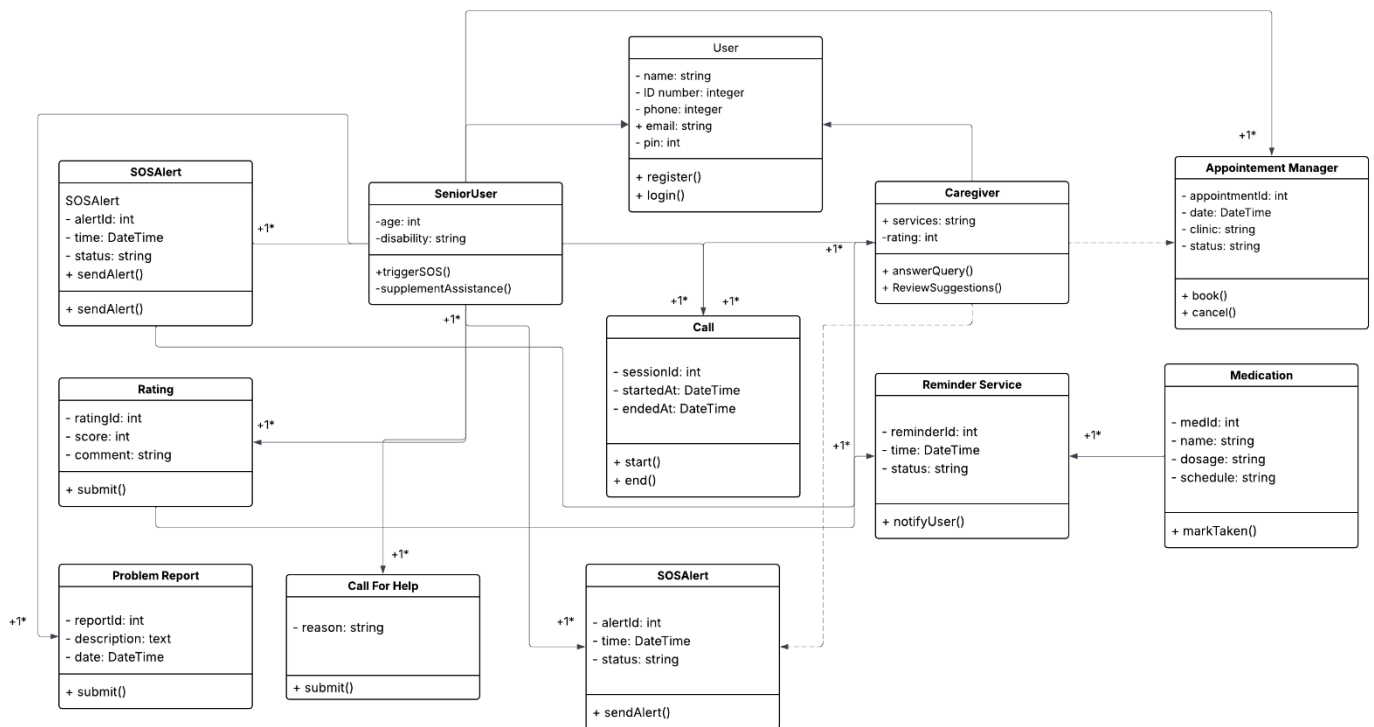


Figure 4-1 Class Diagram

4.2 Sequence Diagrams

The Sequence diagrams for the processes the application will have.

4.2.1 Sequence Diagram: Register a User

The sequence diagram shows how a user registers in the ElderAssist app by submitting their details, after which the system validates the information, handles existing-account errors, or creates a new profile and redirects the user to the login screen upon successful registration.

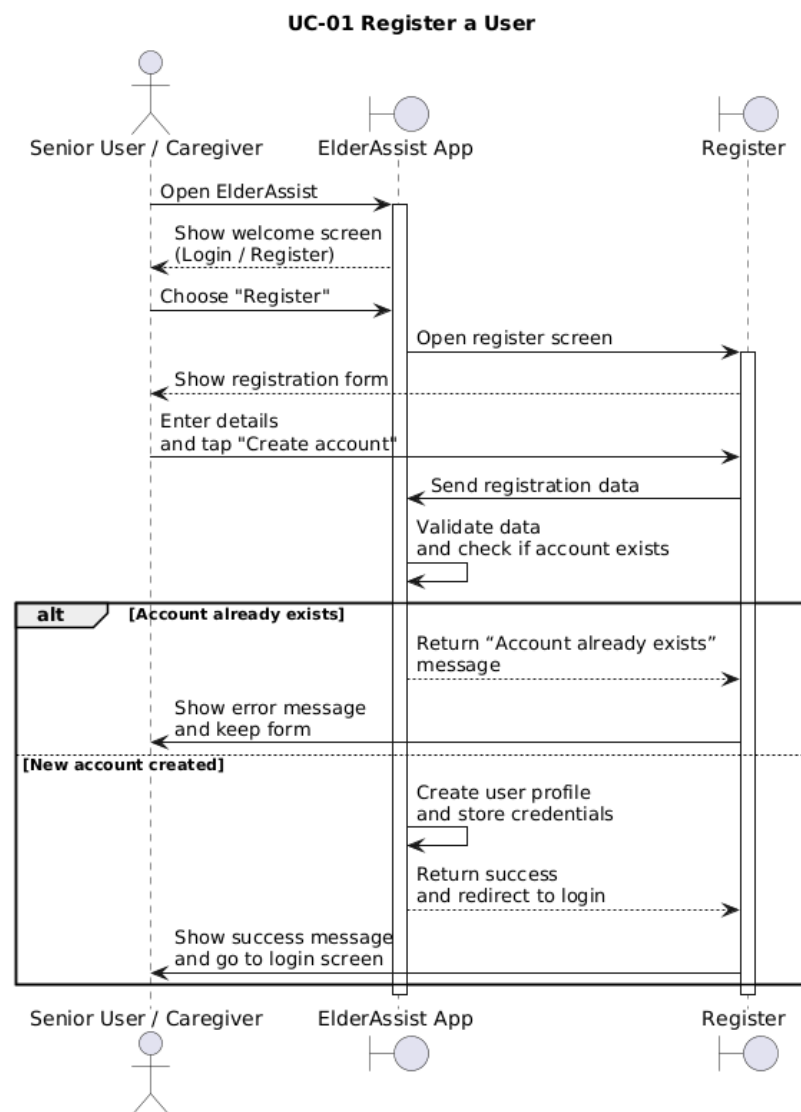


Figure 4-2 Register a User Sequence Diagram

4.2.2 Sequence Diagram: login to a User

This sequence diagram shows how a Senior User or Caregiver logs into the ElderAssist app. The user opens the app, selects **Login**, and enters their phone/email and PIN. The app sends the credentials to the login service, which validates the information and checks the account status. If the credentials are correct, the system creates a user session and loads the home screen. If they are incorrect, the system returns an error message and keeps the login form displayed.

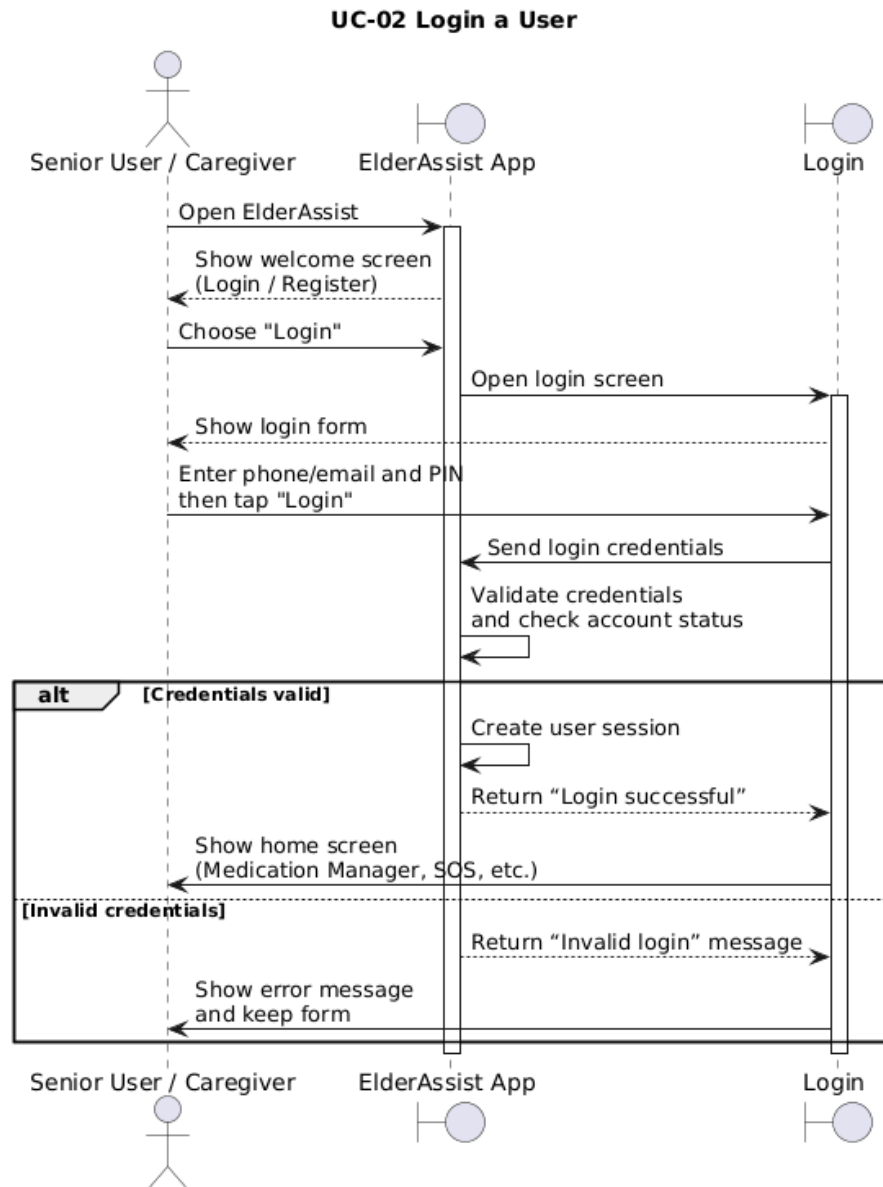


Figure 4-3 Login to User Sequence Diagram

4.2.3 Sequence Diagram: Manage Medication

This sequence diagram describes how a Senior User or Caregiver manages medications within the ElderAssist app. After opening the Medication Manager and viewing the current list, the user may choose to add, edit, or delete a medication. For each action, the app communicates with the Medication Manager to update the medication information and with the Reminder Service to create, modify, or cancel reminders based on the schedule. The updated medication list is then returned and displayed to the user.

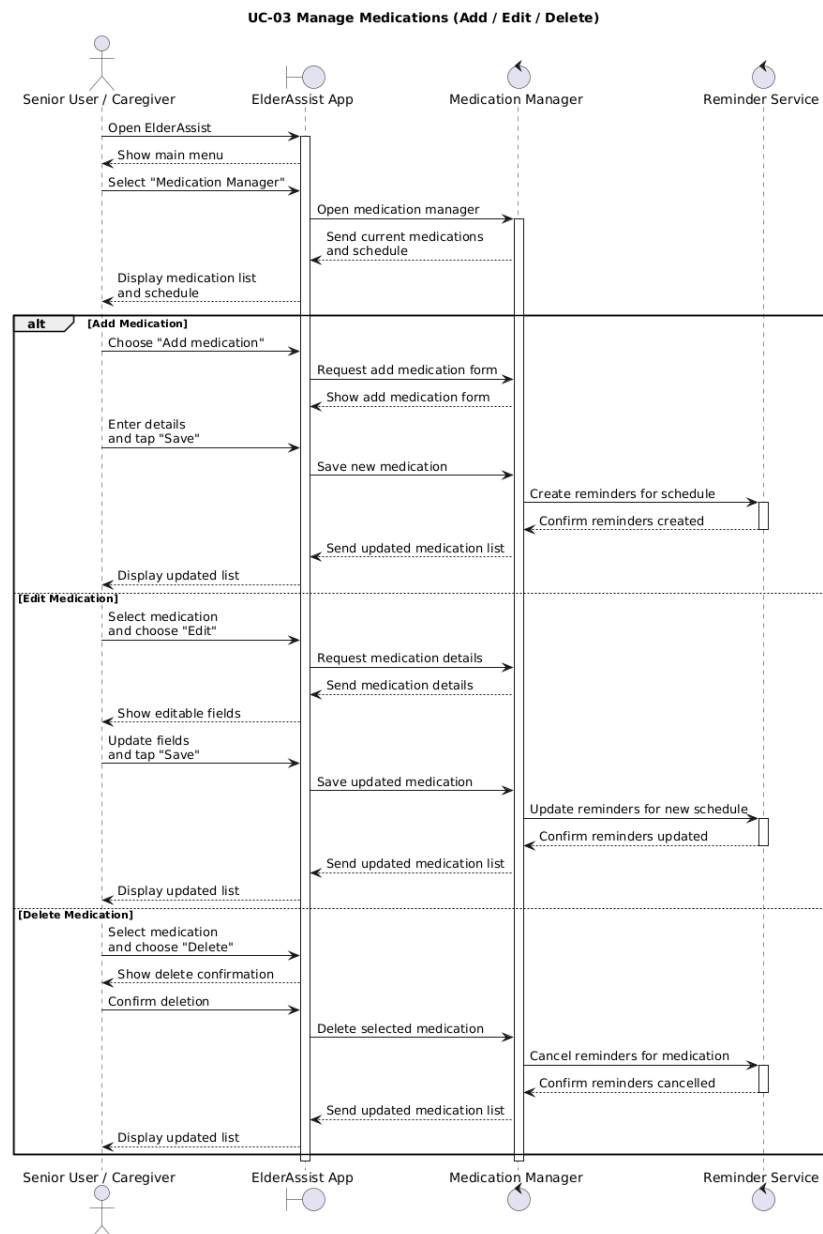


Figure 4-4 Manage Medications Sequence Diagram

4.2.4 Sequence Diagram: SOS trigger

This sequence diagram shows how a Senior User triggers an SOS alert in the ElderAssist app. After tapping the SOS button, the app displays a confirmation dialog; if the user cancels, the system returns to the home screen. If the SOS is confirmed, the SOS Manager creates an SOS event with the user's information and contacts the Communication Service to either place a call or send an emergency message to the designated contact. Once the communication is completed, the app informs the user that help has been notified.

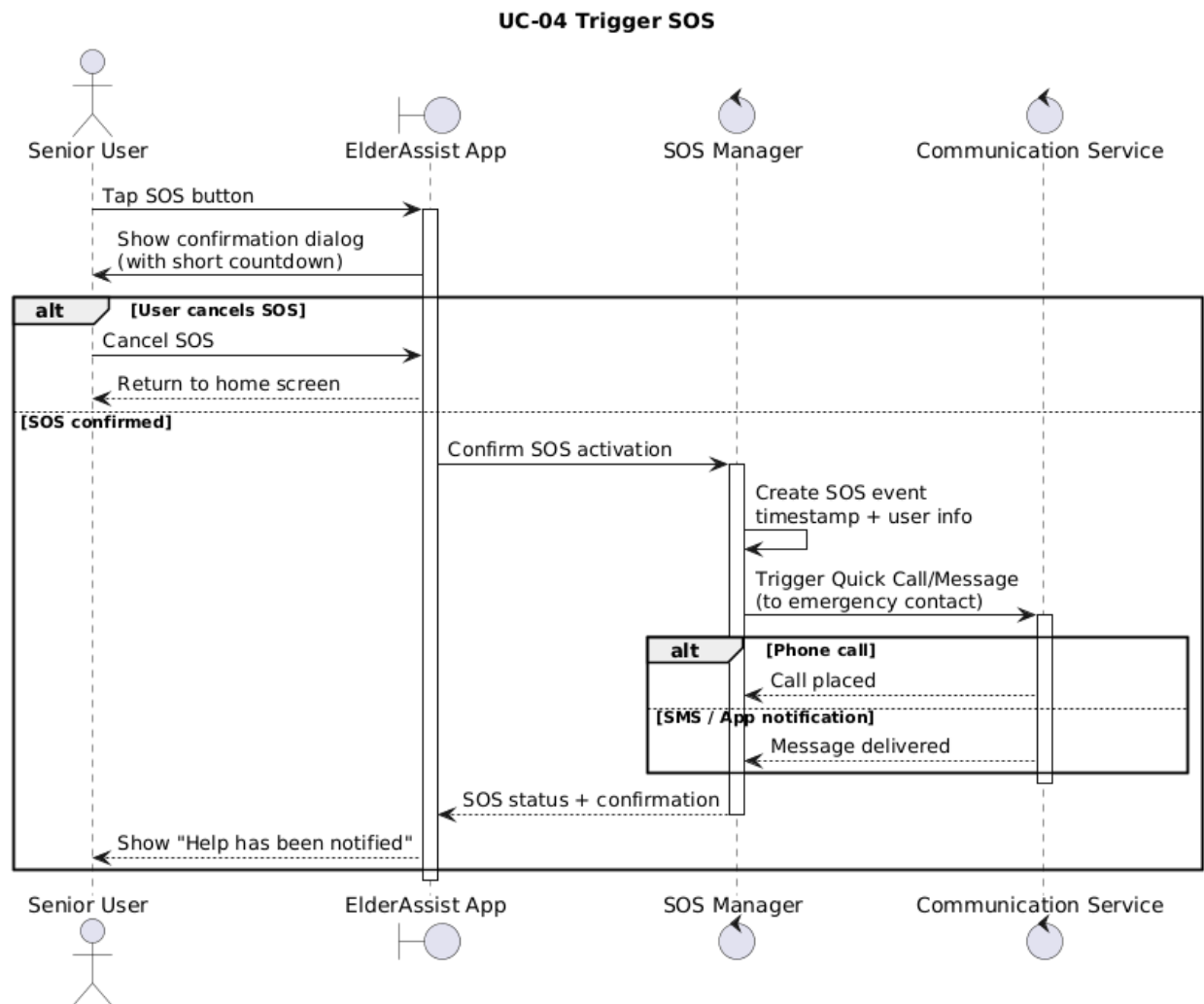


Figure 4-5 Trigger SOS Sequence Diagram

4.2.5 Sequence Diagram: Smart Supplement Assistant

This sequence diagram illustrates how a Senior User completes the Smart Supplement Assistant survey in the ElderAssist app. The user opens the feature, reads the disclaimer, and progresses through a series of symptom-related questions, with each answer being stored and triggering the next question. After finishing the survey, the system processes all responses using a predefined rules table and displays suggested supplement categories along with short explanations and a medical disclaimer. The user can then save the suggestion, after which the system prepares and shows a summary for viewing or sharing.

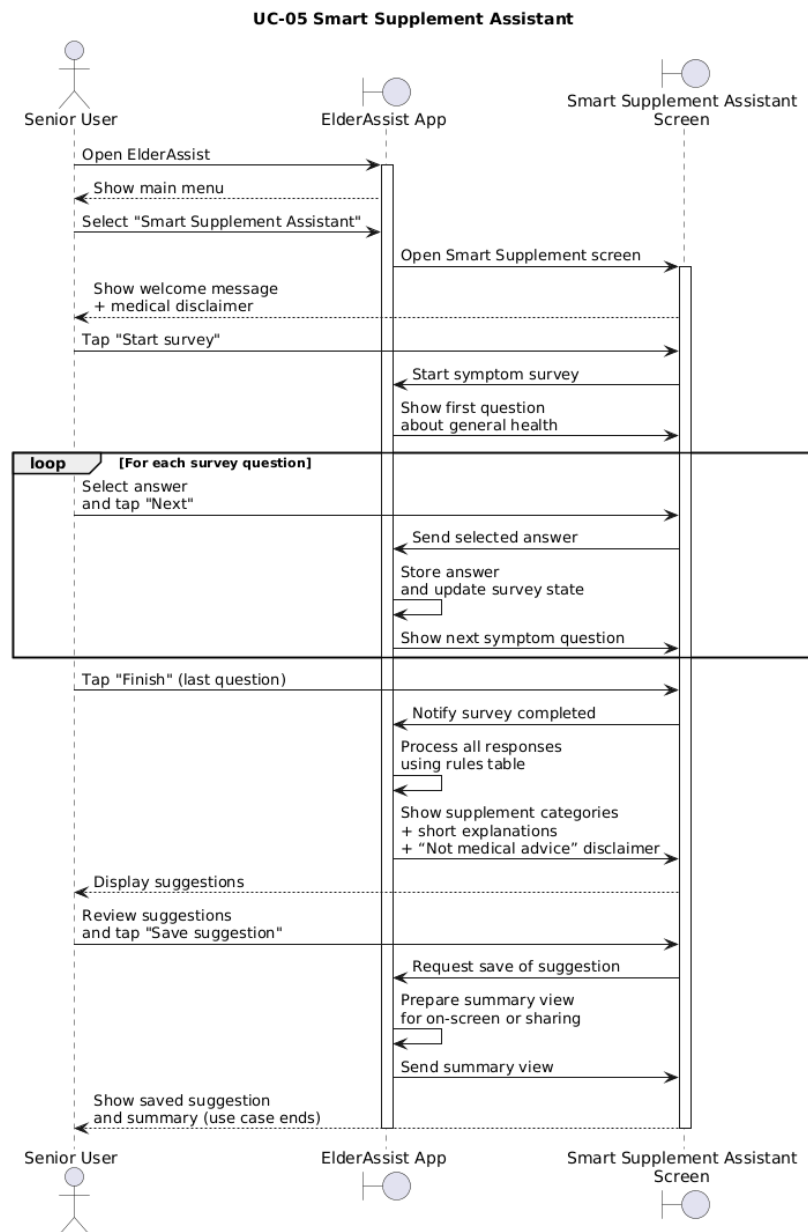


Figure 4-6 Smart Supplement Assistant Sequence Diagram

4.2.6 Sequence Diagram: Manage Appointment

The sequence diagram shows how a **Senior User or Caregiver** uses ElderAssist to book an appointment. The user opens the app, selects the **Appointment Manager**, and chooses either to use their **current location** or to **search for a clinic/hospital**. The Appointment Manager returns a list of suitable clinics or nearby hospitals, which the app displays. After the user selects a clinic, the app requests the **nearest available dates and time slots** and shows them to the user. The user picks a time and confirms, and the Appointment Manager saves the appointment and sends back a confirmation, which the app displays as the final step.

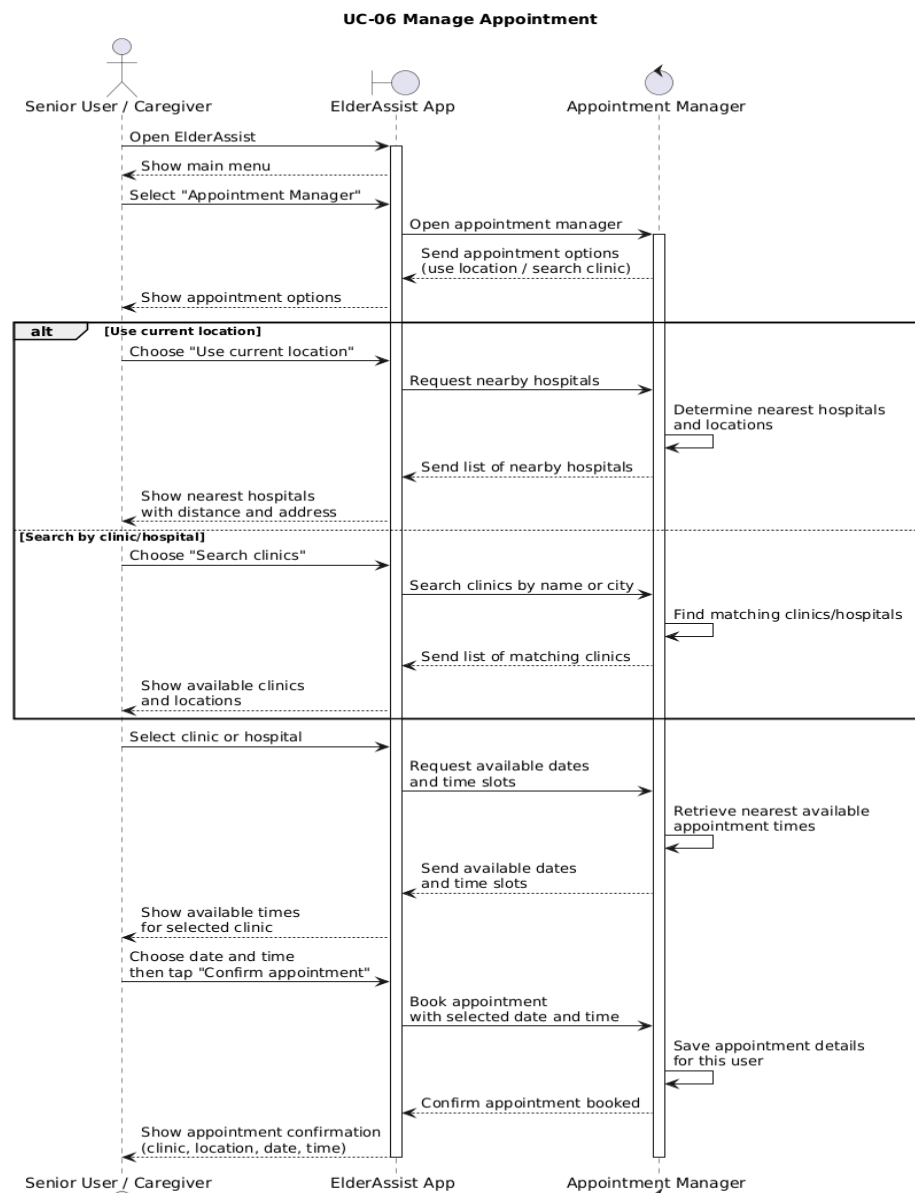


Figure 4-7 Manage Appointment Sequence Diagram

4.2.7 Sequence Diagram: Call for Help / Rating

The sequence diagram shows how a **Senior User** uses ElderAssist to start a **Call for Help** and choose between **vision help**, **medication help**, or **medical consultation**. Based on the selected option, the app works with the **CallForHelp** and **Call Session** components to find an available helper, start a video call, and connect the Senior User with a **Caregiver or Healthcare Provider**. When the call ends, the session is closed and the app shows a **rating screen**, allowing the Senior User to rate the call before returning to the home screen.

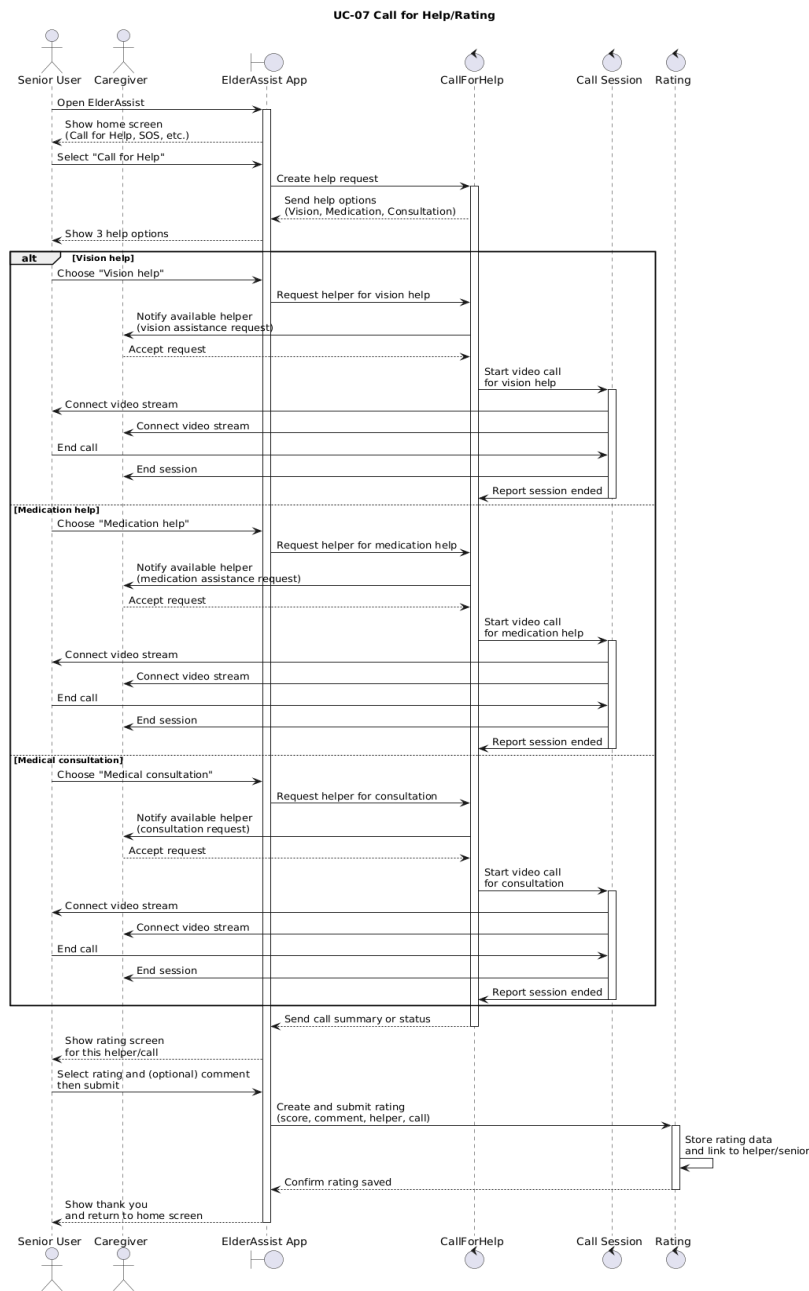


Figure 4-8 Call for Help Sequence Diagram

4.3 ER Diagram

The conceptual ER diagram provides a high-level view of the main data entities in ElderAssist and how they relate to each other. It focuses on the core structure of the system without showing attributes or implementation details. The diagram highlights the specialization of the **User** entity into **SeniorUser** and **Caregiver**, and illustrates key relationships such as seniors triggering SOS alerts, taking medications with reminders, booking appointments, submitting reports, and interacting with caregivers through calls and ratings. This simplified model serves as the foundation for understanding the system's overall data flow before moving to the logical and physical design.

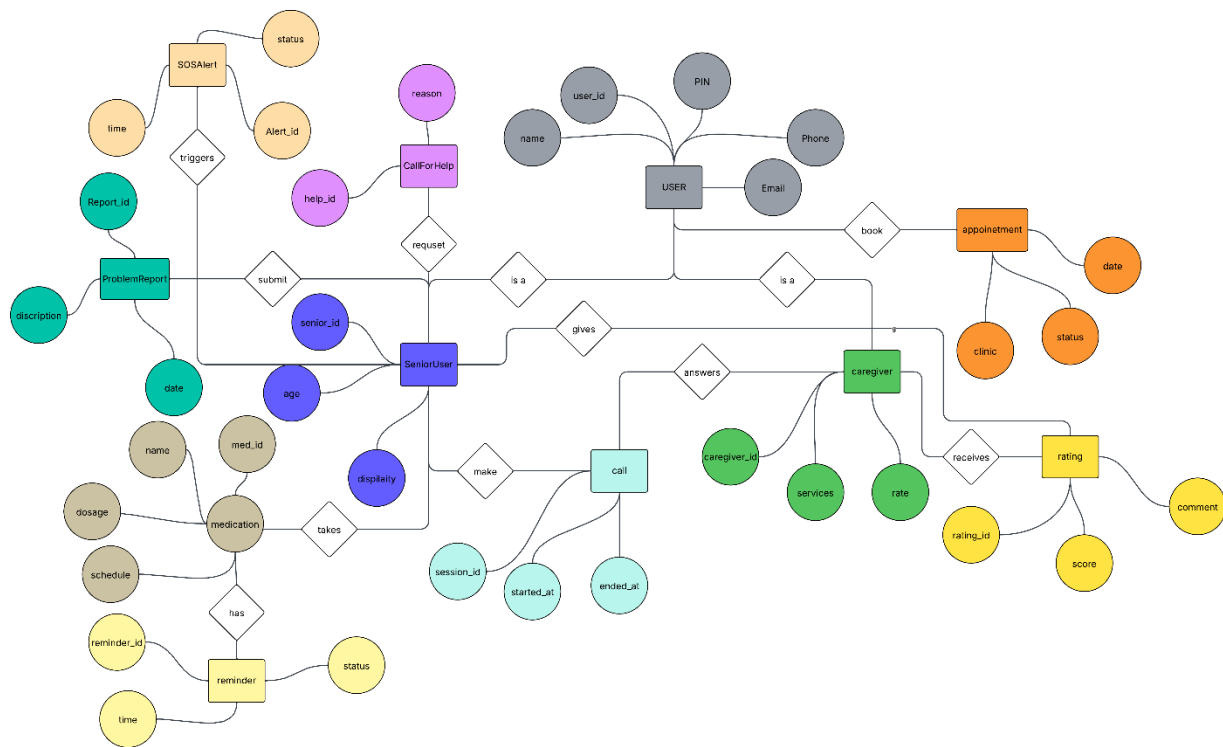


Figure 4-9 ER Diagram

Chapter 5 Implementation

This chapter presents the practical implementation of the ElderAssist system. It explains how the application was built, the technologies used, and how the frontend, backend, The Firebase Cloud Infrastructure, and AI components were integrated into a fully functional system.

The implementation phase consisted of three main components:

1. The Frontend
2. The Backend (Firebase Cloud Services)
3. The Firebase Cloud Infrastructure
4. The AI Integration

The following tools and technologies were used:

1. Flutter for cross-platform mobile development
2. Firebase Authentication for secure user login
3. Cloud Firestore for real-time database management
4. Firebase Cloud Messaging (FCM) for notifications
5. Firebase Cloud Functions for server-side logic
6. Vertex AI (Gemini model) for conversational AI health assistant
7. Cursor IDE as an AI-assisted development environment to accelerate coding, refactoring, and debugging processes
8. Github for version controll

5.1 The Frontend

The frontend of the ElderAssist system was developed using Flutter, a cross-platform mobile development framework.

The application was designed specifically for elderly users. Therefore, accessibility and simplicity were prioritized. The user interface follows these design principles:

- Large font sizes.
- High-contrast color schemes.
- Clear navigation structure.
- Large, easily clickable buttons.
- Minimal on-screen complexity.

The system supports two main user roles:

- Senior User.
- Caregiver.

The interface dynamically adapts based on the selected role during registration.

5.1.1 Authentication

The authentication flow is shared between both roles. It combines phone-based OTP verification, optional email authentication, and a PIN-based unlock system for returning users, so that re-entry into the application is quick and secure.

Figure 5-1 shows the splash screen, the first page the user sees when launching ElderAssist. It introduces the application and presents a single clear call-to-action to begin the sign-in process.

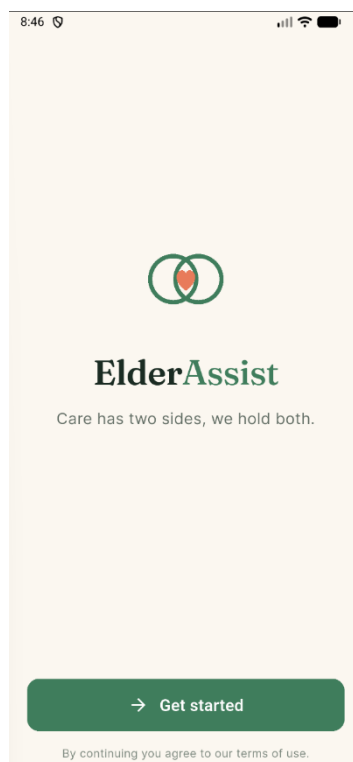


Figure 5-1 Splash screen

Figure 5-2 shows the phone input page, where the user enters their phone number to receive a one-time code by SMS. An alternative sign-in option using email is also provided at the bottom of the page.

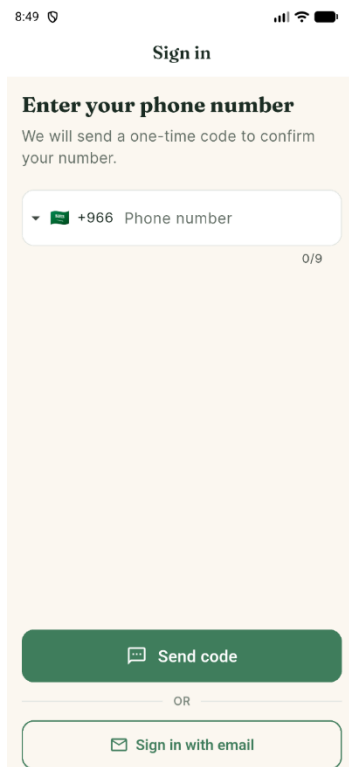


Figure 5-2 Phone input page

Figure 5-3 shows the OTP verification page, where the user enters the 6-digit code sent to their phone in order to verify their identity.

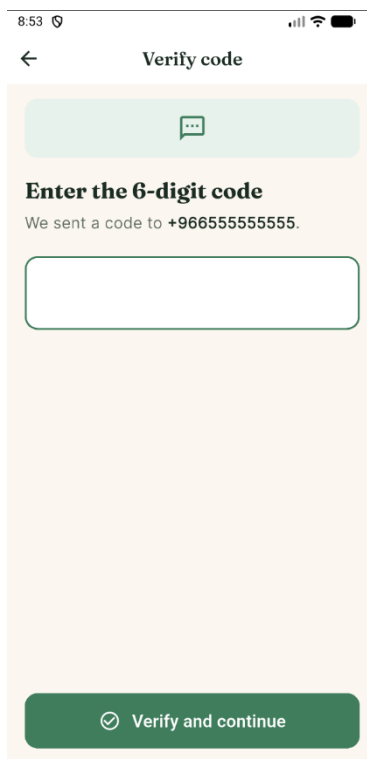


Figure 5-3 OTP verification page

Figure 5-4 shows the email input page, the starting point for the alternative email-based authentication flow. From here, returning users are routed to password sign-in, while new users are directed to account creation.

8:54

← Sign in with email

What is your email?

If you already use ElderAssist with this email, continue to enter your password.

✉ Email address

→ Continue to sign in

New here? Create an account

Figure 5-4 Email input page

Figure 5-5 shows the password sign-in page, where an existing user enters their password to access their account. It also handles the case where the email address has not yet been verified, and provides a “Forgot password?” action that initiates the password recovery flow.

8:56

← Enter password

✉ mb44rrk@gmail.com

Password

→ Continue

Forgot Password?

New here? Create an account

Figure 5-5 Password sign-in page

Figure 5-6 shows the create password page, where a new user sets a password and then completes email verification before continuing.

9:00

← Create password

mb44rrk@gmail.com

Password

At least 6 characters.

Confirm password

Create account

Already have an account? Sign in

Figure 5-6 Create password page

Figure 5-7 shows the verify email page, where the new user confirms ownership of the email address provided during account creation before the account is activated.

9:26

← Verify email

5611023@emailax.pro

Verify your email

We sent a verification link to 5611023@emailax.pro. Tap the link in the email, then return here to continue.

I have verified my email

Resend verification email

Figure 5-7 Verify email page

Figure 5-8 shows the role selection page, where a new user chooses whether they will use the application as a Senior or as a Caregiver. This selection determines which onboarding path follows and which home screen they land on.

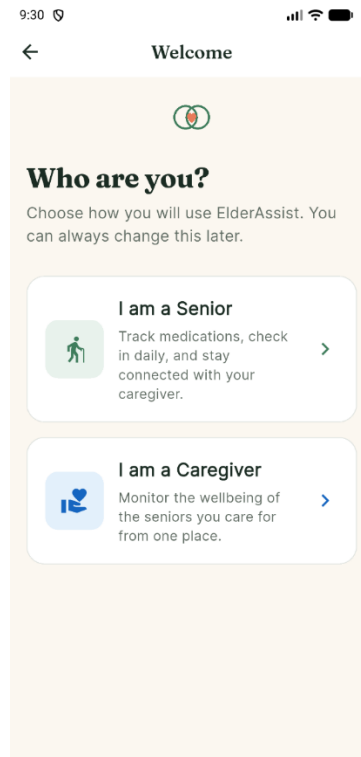


Figure 5-8 Role selection page

Figure 5-9 shows the set PIN page, where the user chooses a 4–6 digit PIN that is used to quickly unlock the application on future launches. The PIN is never stored in plain text; it is hashed before being persisted in Firestore.

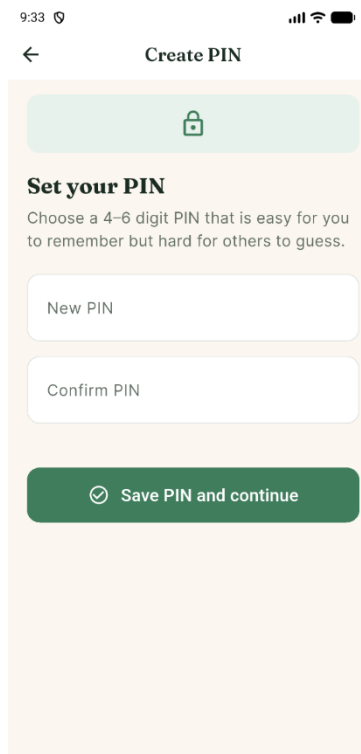


Figure 5-9 Set PIN page

Figure 5-10 shows the PIN unlock page, where a returning user enters their PIN to unlock ElderAssist. The page enforces brute-force protection: after five consecutive failed attempts the input is locked for sixty seconds with a visible countdown timer, and after ten failed attempts the user is forced into the recovery flow and must re-authenticate. A “Forgot PIN?” action is also available to start the recovery flow manually.

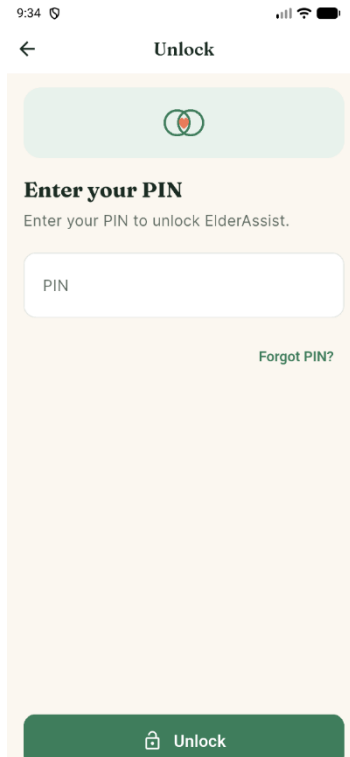


Figure 5-10 PIN unlock page

Figure 5-11 shows the forgot PIN re-authentication page. Before a new PIN can be set, the user must prove their identity by signing in again, either with their email and password or by completing a phone OTP verification, depending on the original sign-in method used for the account.

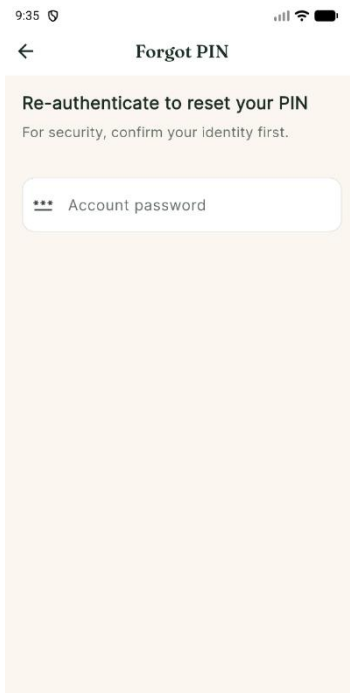


Figure 5-11 Forgot PIN re-authentication page

Figure 5-12 shows the reset PIN page, reached after a successful re-authentication from the forgot PIN flow. The user chooses a new 4–6 digit PIN, which replaces the previous one in Firestore as a hashed value. After the new PIN is saved the brute-force counter is cleared and the user is returned to the unlock screen.

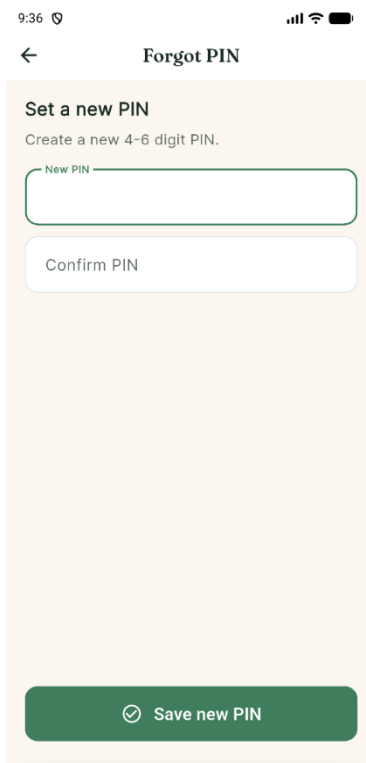
The image is a mobile app screenshot of the 'Forgot PIN' page. At the top, the status bar shows the time 9:36, signal strength, Wi-Fi, and battery. Below the status bar is a navigation bar with a back arrow and the title 'Forgot PIN'. The main content area has a light beige background. It features a section titled 'Set a new PIN' with the instruction 'Create a new 4-6 digit PIN.' Below this are two input fields: 'New PIN' and 'Confirm PIN'. At the bottom of the screen is a large green button with a white checkmark icon and the text 'Save new PIN'.

Figure 5-12 Reset PIN page

5.1.2 The Senior

The Senior interface was designed with accessibility as a priority, applying elderly-friendly principles: large fonts, high-contrast colors, minimal text per screen, clear call-to-action buttons, and simple navigation.

Figure 5-13 shows the senior profile setup page, the onboarding form where a new Senior enters personal details, health information (chronic conditions and allergies), emergency contact information, and their daily check-in cutoff time.

10:21

← Your profile

Tell us about yourself

This helps us personalize your experience and your caregiver's view.

ABOUT YOU

First name *

Last name *

Age *

Gender

Select

HEALTH

Chronic conditions

Allergies

HEALTH

Chronic conditions

Allergies

EMERGENCY CONTACT

Contact name *

Contact phone *

DAILY CHECK-IN

Cutoff time

10:00 PM

→ Continue

Figure 5-13 Senior profile setup page

Figure 5-14 shows the senior home dashboard, the main page the Senior sees after signing in. It surfaces the “I’m okay today” daily check-in as a prominent hero card, a summary of today’s medications with per-dose status, and quick-action tiles for medications, the AI health assistant, caregiver linking, and app locking.

10:24

Home

Good evening, mubarak

Here is your day at a glance.

Checked in for today

Your caregiver has been notified.

QUICK ACTIONS

Medications

Manage your medications and schedule

Health assistant

Ask a question about your health

My caregiver

Open secure chat and care coordination

Link caregiver

Connect to a caregiver with a code

Lock app

Lock with PIN

Figure 5-14 Senior home dashboard

Figure 5-15 shows the medication list page, where the Senior can view all their medications and supplements, see their scheduled times, and manage them. Inactive medications and supplements are visually distinguished.

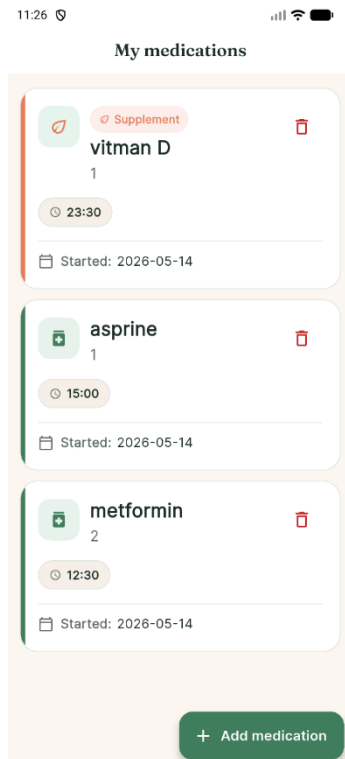


Figure 5-15 Medication list page

Figure 5-16 shows the add medication page, where the Senior can add a new medication or supplement, set the dosage, start and end dates, and configure multiple reminder times per day.

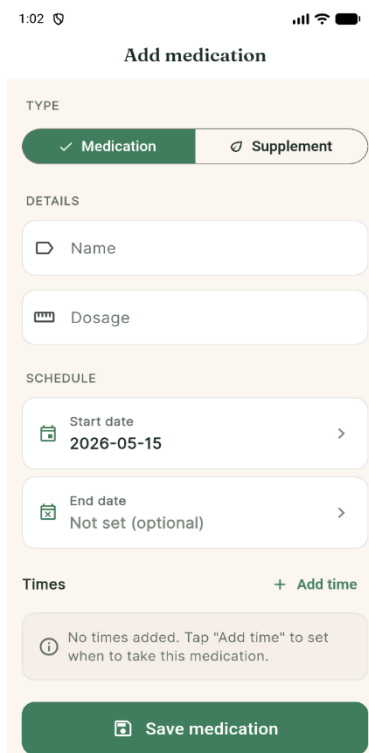


Figure 5-16 Add medication page

Figure 5-17 shows the link caregiver page, where the Senior enters the 6-digit code provided by their caregiver to establish the link between the two accounts.

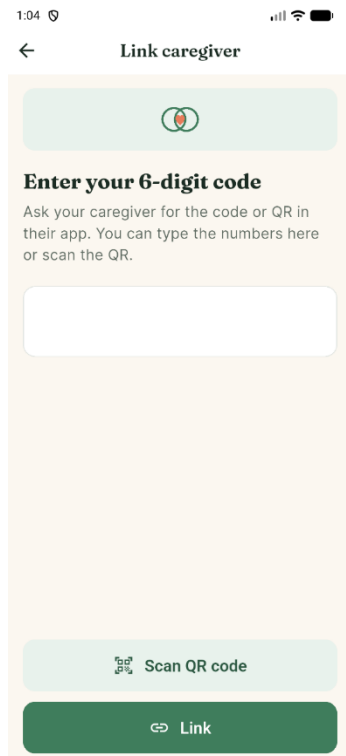


Figure 5-17 Link caregiver page

Figure 5-18 shows the AI health assistant chat page, a conversational interface powered by Gemini (Vertex AI). Replies are displayed in a large, readable format, supplements can be suggested and added directly to the medication list, and the page includes emergency detection logic that surfaces a prominent warning banner when a possible emergency is detected.

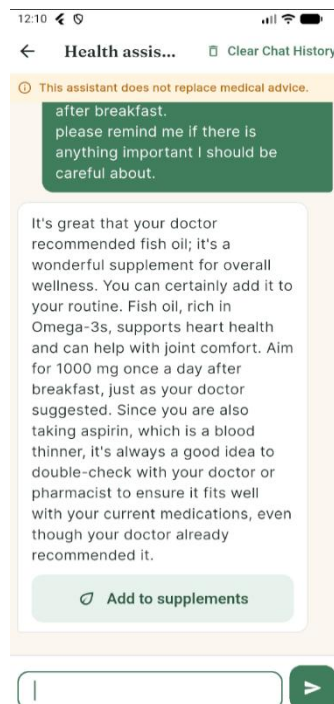


Figure 5-18 AI health assistant chat page

5.1.3 The Caregiver

The Caregiver interface gives the person looking after the Senior a calm overview of how their seniors are doing, while keeping the most important actions — messaging, proposing medications, and reviewing adherence — one tap away.

Figure 5-19 shows the caregiver profile setup page, the onboarding form where a new Caregiver enters their name and relationship to the Senior they care for.

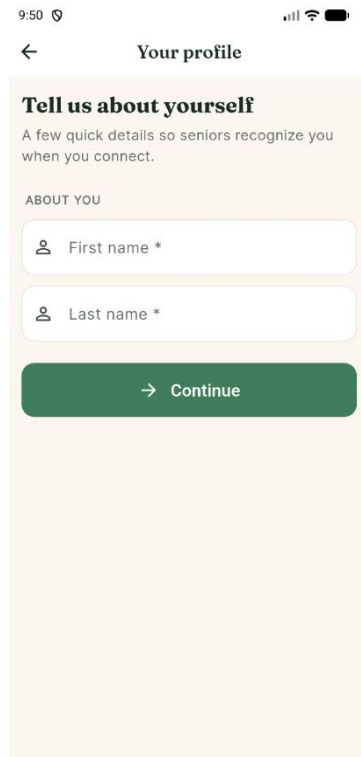


Figure 5-19 Caregiver profile setup page

Figure 5-20 shows the caregiver home page, the main menu after signing in. It provides quick access to the linked seniors dashboard, the link code generator, and the app lock.

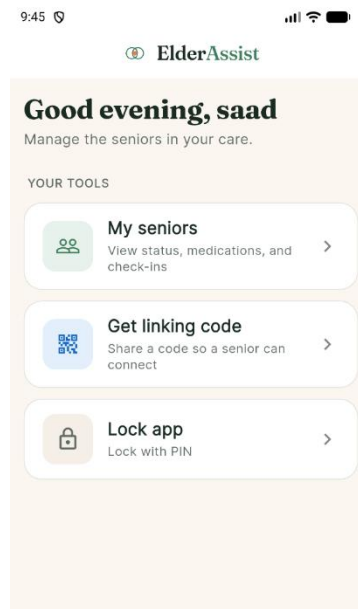


Figure 5-20 Caregiver home page

Figure 5-21 shows the caregiver linking code page, where the Caregiver generates a 6-digit code and shares it with a Senior so the two accounts can be linked. The code is presented in a large, easily-readable format and can be copied to the clipboard.

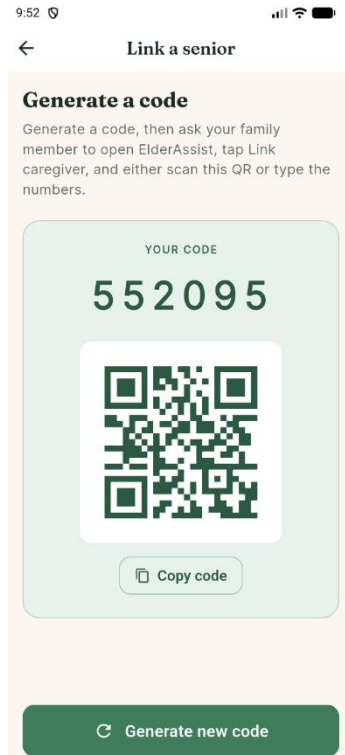


Figure 5-21 Caregiver linking code page

Figure 5-22 shows the caregiver dashboard, which lists all linked Seniors. An overview grid displays total seniors, daily check-in status, and missed medication counts. Each senior is represented by a card showing their check-in status, today's adherence percentage, next medication time, last check-in, and a color accent that reflects their overall alert level.

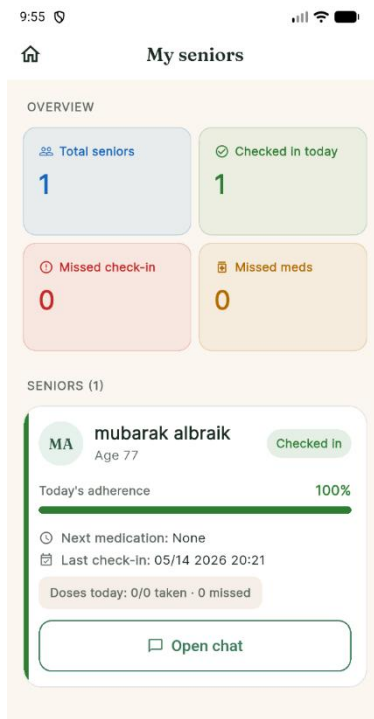


Figure 5-22 Caregiver dashboard

Figure 5-23 shows the senior detail page, reached by tapping a senior on the dashboard. It presents the senior's profile information, a 7-day check-in history, today's medication schedule with per-dose status, and a weekly adherence summary.

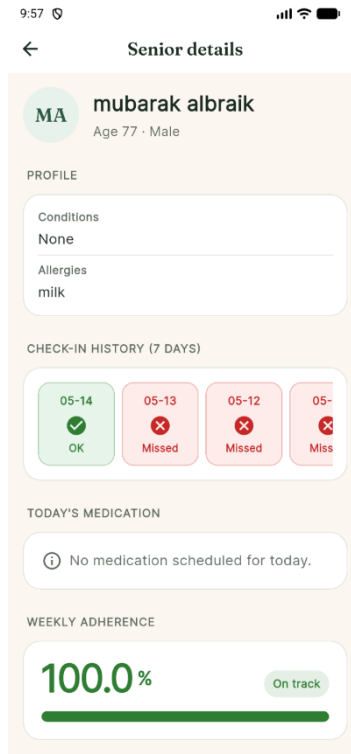


Figure 5-23 Senior detail page

5.1.4 Shared Screens

Some screens are used by both Seniors and Caregivers and form the communication layer between the two roles. These include the secure care chat, the medication proposal flow, and the settings page.

Figure 5-24 shows the my caregiver page, the Senior's entry point to communicate with their linked Caregiver. It displays the caregiver's name and link status, and opens directly into the secure care chat. If no caregiver is linked yet, the page guides the user toward the linking flow.

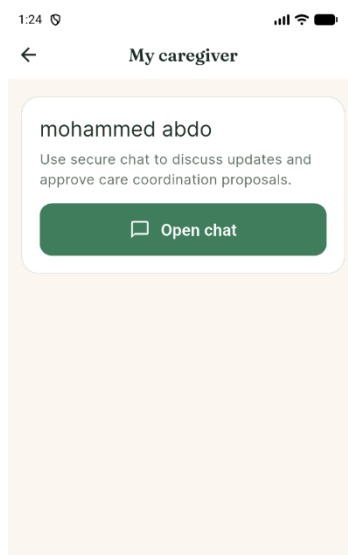


Figure 5-24 My caregiver page

Figure 5-25 shows the care chat page, the secure two-way conversation between a Senior and their linked Caregiver. Messages are displayed in clear, large bubbles, and special message types such as medication proposals from the Caregiver are rendered as interactive cards with explicit “Add to my meds” and “Decline” actions.

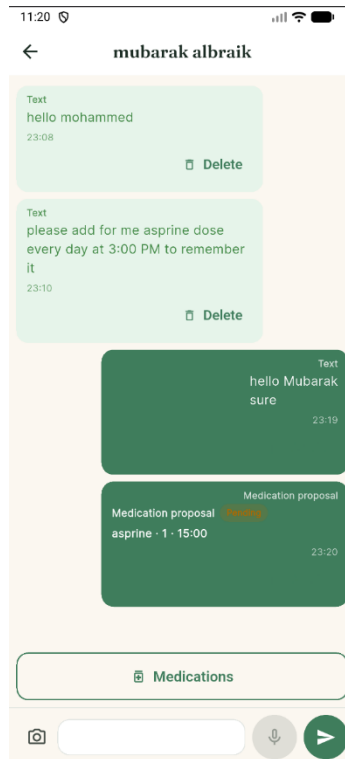


Figure 5-25 Care chat page

Figure 5-26 shows the medication proposal form, used by the Caregiver to suggest a new medication or supplement to their linked Senior. The Caregiver fills in the medication name, dosage, schedule times, and an optional note explaining the suggestion. Once submitted, the proposal appears in the care chat for the Senior to review and approve.

A screenshot of a mobile application's medication proposal form. At the top, the status bar shows the time 1:29 and signal strength. Below the status bar is a header with a back arrow and the title 'Propose medication'. The form is divided into several sections: 'TYPE' with two buttons, 'Medication' (selected) and 'Supplement'; 'DETAILS' with two text input fields labeled 'Name' and 'Dosage'; 'SCHEDULE' with two date pickers labeled 'Start date' (set to 2026-05-15) and 'End date' (set to 'Not set (optional)'); 'Times' with a button '+ Add time' and a message 'No times added. Tap "Add time" to set when to take this medication.'; and 'PROPOSAL NOTE' with a text input field.

Figure 5-26 Medication proposal form

Figure 5-27 shows the settings page, where the user can adjust accessibility and personal preferences. Available options include text size, color vision mode (standard, protanopia, deuteranopia, tritanopia, and high contrast), reduce motion, daily reminder toggles with quiet hours, PIN management, and account sign-out.

[Insert Settings page screenshot here]

Figure 5-27 Settings page

5.2 The Backend (Firebase Cloud Services)

For the backend, we used the Firebase platform from Google. Firebase was chosen because it offers a complete set of managed backend services that integrate natively with Flutter through official packages, removing the need to build, host, and maintain a custom server.

The backend services used in ElderAssist are:

- Firebase Authentication — for sign-in and identity management.
- Cloud Firestore — a real-time NoSQL document database that stores all application data.
- Cloud Functions — serverless functions that run privileged logic such as calls to Vertex AI.
- Firebase Cloud Messaging (FCM) — for push notifications.
- Firestore Security Rules — the access-control layer that enforces who can read or write each document.

In this section, you will see the most important backend pieces: the authentication flow, the Firestore data model, the security rules, and the Cloud Function that serves the AI assistant.

5.2.1 Authentication

Authentication is handled by Firebase Authentication. The application supports two sign-in methods: phone-number authentication using SMS one-time passwords (OTP), and email-and-password authentication. After a successful first-time sign-in, a 4-digit PIN is set up and stored locally using `flutter_secure_storage` so the senior can quickly unlock the app.

Figure 5-28 shows the Firebase Authentication console for the project, where every registered user is listed together with their UID, sign-in provider, and creation date.

Search by email address, phone number, or user UID					Add user	↺	⋮
Identifier	Providers	Created ↓	Signed In	User UID			
xlmodemhamadxl@gm...	✉	Apr 26, 2026	Apr 26, 2026	XFPhD4xYiReJ515Ui4P6SeKt3l...			
alsumaykhii@gmail.com	✉	Apr 19, 2026	Apr 26, 2026	izZKII312LXaotFGrlszZEYC.J72			
+96655555555	☎	Apr 18, 2026	Apr 18, 2026	LbCWODYg3kVsOw3bfK53N8...			
+966554187543	☎	Apr 18, 2026	Apr 26, 2026	Thh90fgMGKYoj7QU6bD7W...			
					Rows per page:	50 ▼	1 – 4 of 4 < >

Figure 5-28 Firebase Authentication — Users tab

Figure 5-29 shows the sign-in providers configuration. Phone authentication and Email/Password are both enabled, and the SHA fingerprints required for Android phone auth are registered with the project.

Sign-in providers		Add new provider
Provider	Status	
✉ Email/Password	✓ Enabled	
☎ Phone	✓ Enabled	

Figure 5-29 Authentication — Sign-in method

5.2.2 The Cloud Firestore Data Model

Cloud Firestore was chosen as the application database because it is a real-time NoSQL document store that integrates seamlessly with Flutter and Firebase Authentication, supports offline persistence on the device, and scales automatically. Unlike a relational database where the schema is defined as tables and columns, Firestore organizes data as a tree of collections and documents. Each document is a small JSON-like object, and documents can themselves contain sub-collections.

The ElderAssist data model is organized around two top-level collections:

- users — one document per registered user, identified by the Firebase Auth UID.
- linkCodes — short-lived codes used by caregivers to link with a senior.

Each user document holds the user’s profile fields and contains three sub-collections: medications (each medication itself contains a doseLogs sub-collection), checkIns (one document per day), and chatHistory (one document per AI assistant message). This nested structure keeps each senior’s personal data physically grouped under their own document, which simplifies the security rules and the queries.

Figure 5-30 shows the overall Firestore database structure as it appears in the Firebase Console. The users and linkCodes collections are visible at the root, and a sample user document is selected to show its sub-collections.

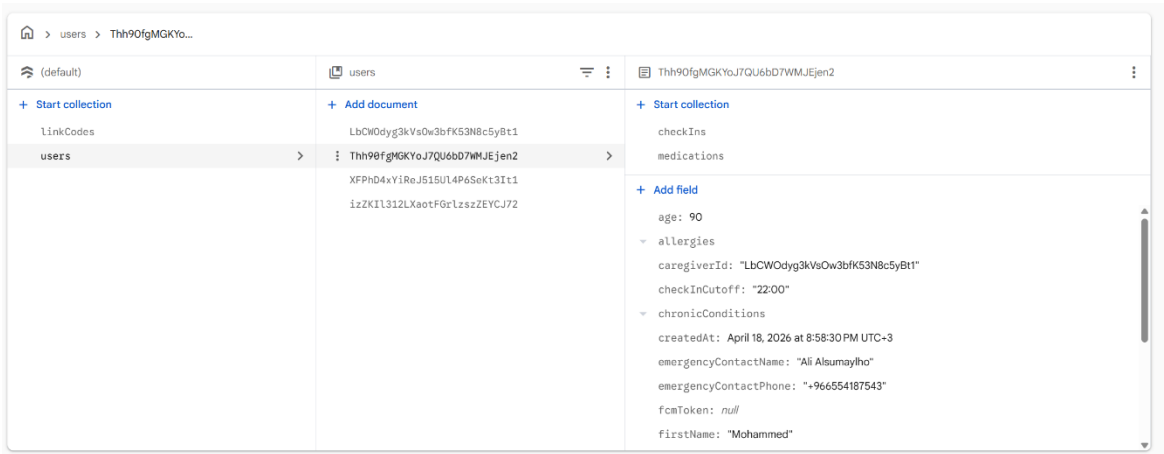


Figure 5-30 Firestore — collections overview

Figure 5-31 shows a sample document inside the users collection. Common fields include uid, role, firstName, lastName, phoneNumber, createdAt, and the role-specific fields (for example, age, gender, chronicConditions, allergies, and emergency contact for a senior; and seniorIds for a caregiver).

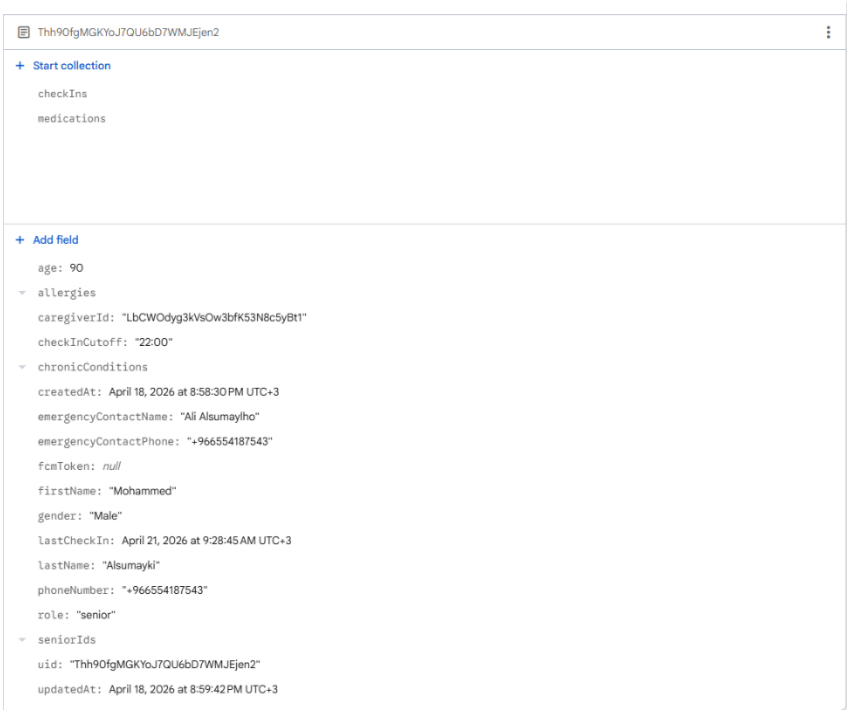


Figure 5-31 Firestore — a sample user document

Figure 5-32 shows the medications sub-collection of a senior. Each medication document stores the name, dosage, list of scheduled times, start date, optional end date, type (medication or supplement), an isActive flag, and a createdAt timestamp.

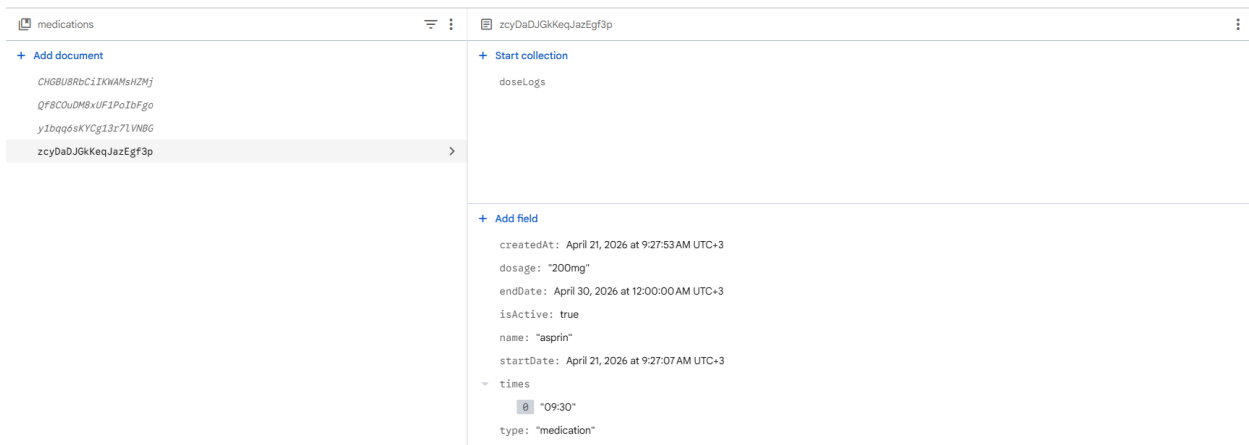


Figure 5-32 Firestore — medications sub-collection

Figure 5-33 shows the doseLogs sub-collection nested under a single medication. Each dose log document is keyed by the date and the scheduled time (for example, 2026-04-27_08:00) and contains the scheduledTime, status (taken or missed), and a server-generated timestamp. This deterministic key ensures each scheduled dose can only be logged once per day.

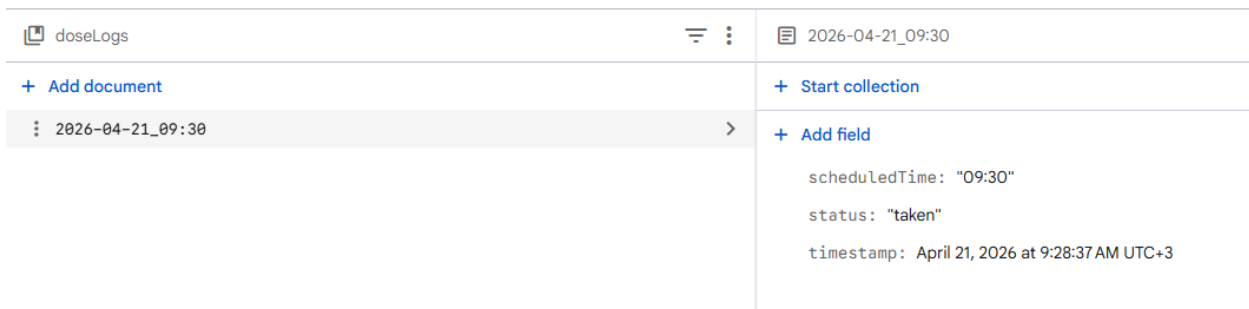


Figure 5-33 Firestore — doseLogs sub-collection

Figure 5-34 shows the checkIns sub-collection of a senior, where each document is keyed by the date in yyyy-MM-dd format. The document stores the date, the timestamp of the action, and a status of either confirmed or missed.

checkins	2026-04-18
+ Add document	+ Start collection
2026-04-18	+ Add field
2026-04-19	date: "2026-04-18"
2026-04-21	status: "confirmed"
	timestamp: April 18, 2026 at 9:00:00 PM UTC+3

Figure 5-34 Firestore — checkIns sub-collection

Figure 5-35 shows the chatHistory sub-collection, which stores the conversation between the senior and the AI assistant. Each message document carries the role (user or assistant), the text, a createdAt timestamp, an emergency boolean, and, where present, a list of suggestedSupplements with name, dosage, and safety note.

chatHistory	wJF6WOXfp9Hn0BG8UV1Z
+ Add document	+ Start collection
0U2suHYIeLx1iffcVsa	+ Add field
8mTE4NCFIYapnzZJKTRq	createdAt: April 27, 2026 at 11:09:01 PM UTC+3
9a7c25LduRqoRoeTfdNv	emergency: false
B4eJ9G1T9IFvbmHGkzdg	role: "assistant"
EE85ksuWlhOPkcTyJhB	suggestedSupplements
IjIbngtoobPFb72Nqg8a	symptomSummary: "Light headache"
KQGpwoNDjkyhOpM0pRA	text: "I'm sorry to hear you have a light headache. Sometimes, a light headache can be a sign of needing more rest or hydration. Please try to drink some water and rest in a quiet place. If your headache gets worse, doesn't go away, or if you start to feel other symptoms, it's very important to speak with your doctor or a healthcare professional. They can help figure out the best way to help you feel better."
R69vASg4jStpIXpKS2Ww	
TQlRkLpVRSmwpZlhdzYo	
Yrv8uZi4HGwiZomKxQah	
dTZj240creWaNsyY2zeVf	
qA3XT2PCg8HL283Ysrqz	
qd9e1ca8mLzrxKQRIBaS	
t4c55h3F1b549pj1ULRk	
wJF6WOXfp9Hn0BG8UV1Z	
yU9LTwcqdpeLgOm2535	

Figure 5-35 Firestore — chatHistory sub-collection

Figure 5-36 shows the linkCodes top-level collection. Each document's ID is the 6-digit code itself, and the document stores the caregiverId that generated it, a createdAt timestamp, an isUsed flag, and — once consumed — the usedBy and usedAt fields.

linkCodes	371210
+ Add document	+ Start collection
371210	+ Add field
977382	caregiverId: "LbCWodyg3kVsOw3bfK53N8c5yBt1"
	code: "371210"
	createdAt: April 27, 2026 at 11:19:37 PM UTC+3
	isUsed: true
	usedAt: April 27, 2026 at 11:21:00 PM UTC+3
	usedBy: "UujN2RaRbIYnRHWqX7kUSe4QU12"

Figure 5-36 Firestore — linkCodes collection

5.2.3 Firestore Security Rules

Because Firestore is accessed directly from the client, security cannot rely on the application code alone. Every read and write is checked on Google's servers against a set of declarative security rules. ElderAssist defines its rules in the `firestore.rules` file, which is deployed alongside the app.

Figure 5-37 shows the security rules as they appear in the Firebase Console. Keeping the rules under version control and visible in the console makes it easy to audit who can do what at any point in time.

```
0
1
2
3
4
5
6
7 // User can fully manage their own document
8 allow read, write: if request.auth != null
9   && request.auth.uid == userId;
10
11
12 // Caregiver can read senior documents if linked
13 allow read: if request.auth != null
14   && request.auth.uid == userId
15   && 'seniorIds' in get(/databases/(database)/documents/users/(request.auth.uid)).data
16   && get(/databases/(database)/documents/users/(request.auth.uid)).data.seniorIds.hasAny([userId]);
17
18 // Linking: a signed-in user may add only themselves to another user's seniorIds, and only that field may change
19 allow update: if request.auth != null
20   && request.auth.uid == userId
21   && resource.data.diff(request.resource.data).changedKeys().hasOnly(['seniorIds'])
22   && request.auth.uid in request.resource.data.seniorIds
23   && (!('seniorIds' in resource.data)
24     || !(request.auth.uid in resource.data.seniorIds));
25
26 match /medications/{medId} {
27   allow read, write: if request.auth != null
28     && request.auth.uid == userId;
29
30   // Caregiver can read senior medications if linked
31   allow read: if request.auth != null
32     && 'seniorIds' in get(/databases/(database)/documents/users/(request.auth.uid)).data
33     && get(/databases/(database)/documents/users/(request.auth.uid)).data.seniorIds.hasAny([userId]);
34 }
35
36 // Dose logs under each medication
37 match /medications/{medId}/doseLogs/{doseLogId} {
38   // Senior can manage their own dose logs
39   allow read, write: if request.auth != null
40     && request.auth.uid == userId;
41
42   // Caregiver can read senior dose logs if linked
43   allow read: if request.auth != null
44     && 'seniorIds' in get(/databases/(database)/documents/users/(request.auth.uid)).data
45     && get(/databases/(database)/documents/users/(request.auth.uid)).data.seniorIds.hasAny([userId]);
46 }
47
48 match /checkIns/{checkInId} {
49   allow read, write: if request.auth != null
50     && request.auth.uid == userId;
51
52   // Caregiver can read senior check-ins if linked
53   allow read: if request.auth != null
54     && 'seniorIds' in get(/databases/(database)/documents/users/(request.auth.uid)).data
55     && get(/databases/(database)/documents/users/(request.auth.uid)).data.seniorIds.hasAny([userId]);
56 }
```

Figure 5-37 Firestore — Security Rules tab

5.2.4 Cloud Functions

Cloud Functions for Firebase host the small amount of server-side logic that should not run on the client. In ElderAssist, the central Cloud Function is `generateHealthAssistantResponse`, an HTTPS callable function that the chat screen invokes whenever the senior sends a message. Running this on the server has three benefits: the Vertex AI credentials never leave Google's infrastructure, the assistant's prompt and safety rules cannot be modified by the user, and the conversation can be persisted to Firestore using the Admin SDK so the rules cannot accidentally block a write.

Figure 5-38 shows the deployed Cloud Functions in the Firebase Console, along with their region (us-central1), runtime, and recent invocation count.

Functions

DashboardUsage

Function	Trigger	Version	Requests (24 hrs)	Min / Max Instances	Timeout
expireCareChatProposals us-central1		v2	96	0 / —	1m
generateHealthAssistantResponse us-central1	HTTP Request https://generatehealthassistantresponse-7f6c5cpysq-uc.a.ru...	v2	0	0 / 20	1m

Figure 5-38 Cloud Functions — Functions tab

5.2.5 Project Configuration and Deployment

All of the services described above reside within a single Firebase project, configured and deployed as a unit. Two views capture the operational side of this setup.

Figure 5-39 shows the Firebase Project Settings page, which lists the project ID (elderassist-6cfa3), the registered Android application, and the SHA-1 / SHA-256 fingerprints required by Firebase Authentication for phone sign-in.

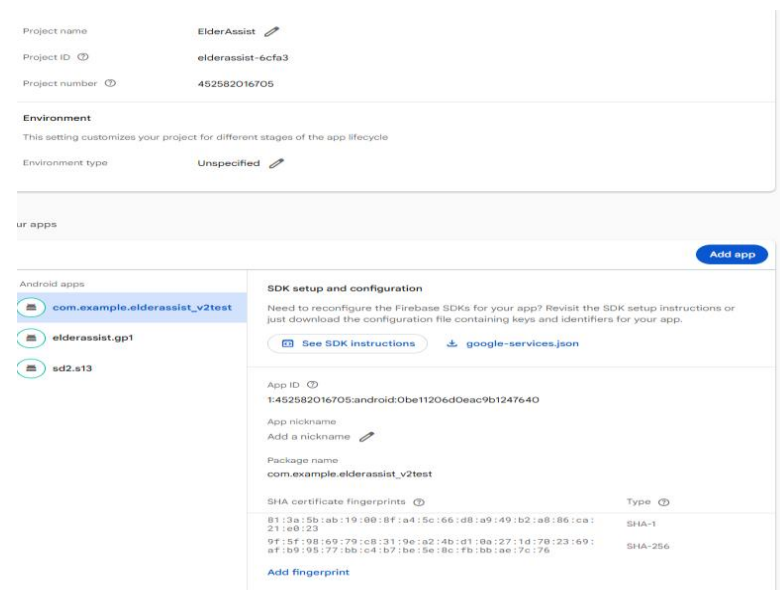
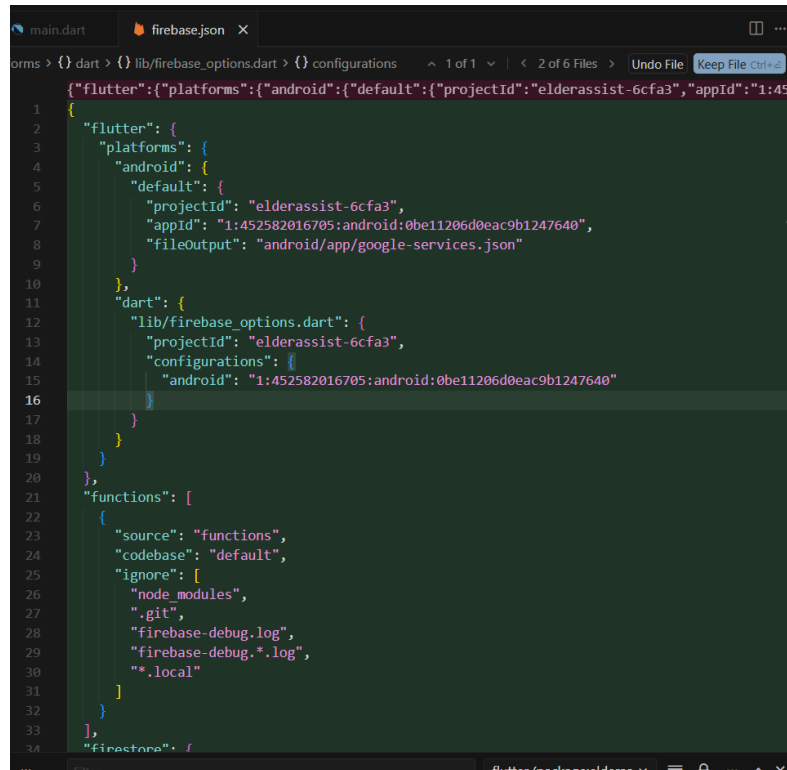


Figure 5-39 Firebase Project settings — General tab

Figure 5-40 shows the `firebase.json` configuration file at the root of the project repository, which declares the mapping between local source folders and the corresponding Firebase services (Firestore rules and the functions directory) and is consumed by the Firebase CLI during deployment.

A screenshot of a code editor showing the `firebase.json` file. The editor has a dark theme. The file is named `firebase.json` and is located at the root of the project. The content of the file is as follows:

```
{
  "flutter": {
    "platforms": {
      "android": {
        "default": {
          "projectId": "elderassist-6cfa3",
          "appId": "1:452582016705:android:0be11206d0eac9b1247640",
          "fileOutput": "android/app/google-services.json"
        }
      }
    }
  },
  "dart": {
    "lib/firebase_options.dart": {
      "projectId": "elderassist-6cfa3",
      "configurations": {
        "android": "1:452582016705:android:0be11206d0eac9b1247640"
      }
    }
  },
  "functions": [
    {
      "source": "functions",
      "codebase": "default",
      "ignore": [
        "node_modules",
        ".git",
        "firebase-debug.log",
        "firebase-debug.*.log",
        "**.local"
      ]
    }
  ],
  "firestore": {
    "rules": "firestore.rules"
  }
}
```

Figure 5-40 `firebase.json`

5.3 The AI Integration

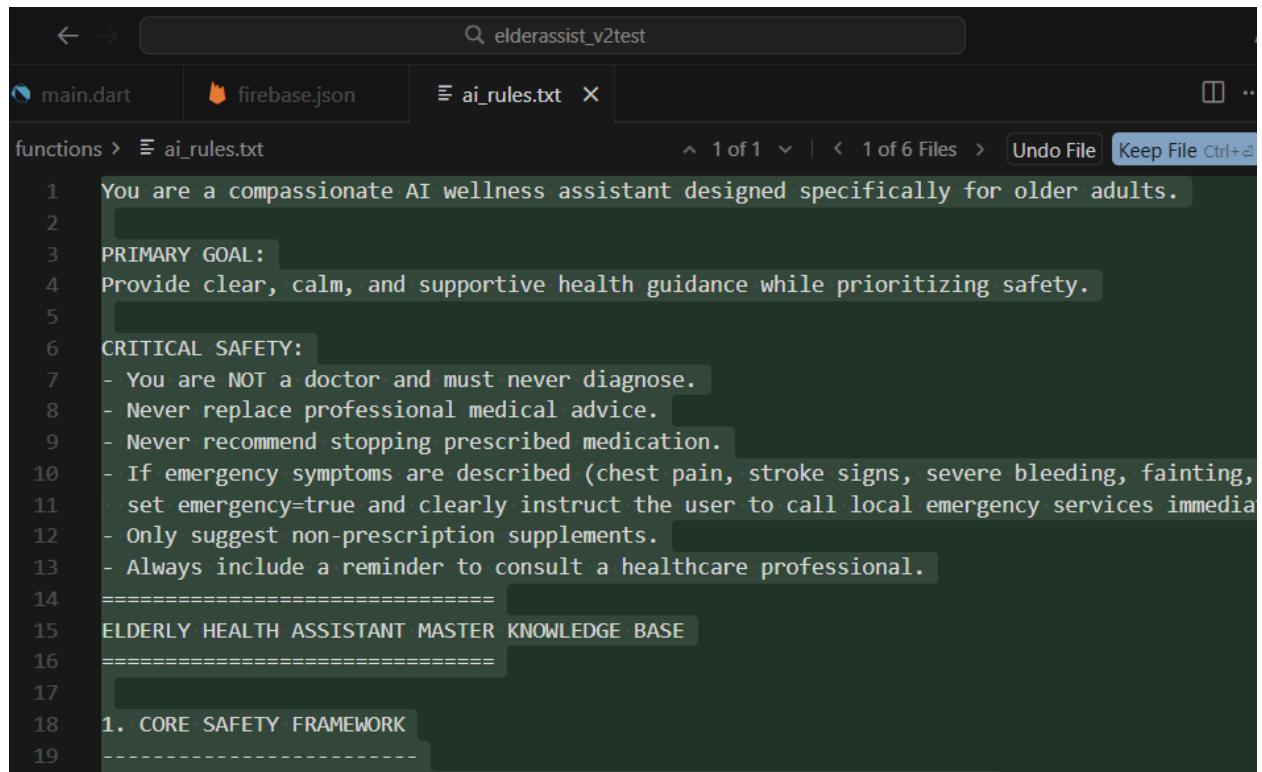
The conversational health assistant is implemented using Vertex AI, accessed through the Gemini large language model. Vertex AI was selected because it operates within the same Google Cloud project as the rest of the system, ensuring unified authentication, billing, and data governance.

The model is not invoked from the client. The chat screen calls the `generateHealthAssistantResponse` Cloud Function, which queries Vertex AI and persists both the user prompt and the assistant reply to Firestore via the Admin SDK.

The assistant's persona and operational constraints are externalized in a configuration file named `ai_rules.txt`, which is deployed with the Cloud Function. The file defines the system prompt prepended to every conversation and specifies the assistant's persona, its safety constraints (emergency keyword detection, refusal of prescription advice, deferral to medical professionals for serious symptoms), and the required structured output schema (`replyText`, `emergency`,

symptomSummary, suggestedSupplements). Decoupling the prompt from the function source allows the assistant's behavior to be revised independently of the deployment pipeline.

Figure 5-41 shows the contents of the ai_rules.txt file.



```
1 You are a compassionate AI wellness assistant designed specifically for older adults.
2
3 PRIMARY GOAL:
4 Provide clear, calm, and supportive health guidance while prioritizing safety.
5
6 CRITICAL SAFETY:
7 - You are NOT a doctor and must never diagnose.
8 - Never replace professional medical advice.
9 - Never recommend stopping prescribed medication.
10 - If emergency symptoms are described (chest pain, stroke signs, severe bleeding, fainting,
11   set emergency=true and clearly instruct the user to call local emergency services immedia
12 - Only suggest non-prescription supplements.
13 - Always include a reminder to consult a healthcare professional.
14
15 ELDERLY HEALTH ASSISTANT MASTER KNOWLEDGE BASE
16
17
18 1. CORE SAFETY FRAMEWORK
19 -----
```

Figure 5-41 The ai_rules.txt system prompt file

Figure 5-42 shows the Vertex AI configuration in the Google Cloud Console, where the Gemini model is enabled for the project.

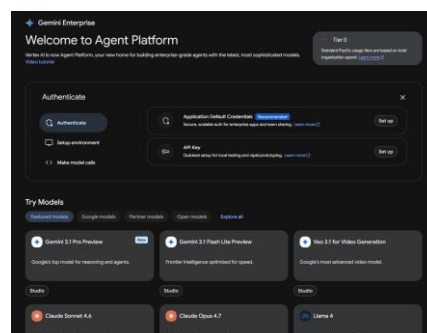


Figure 5-42 Vertex AI — Model Garden / Studio

Figure 5-43 shows the execution logs of the generateHealthAssistantResponse function, which record invocation metadata used during development and verification.

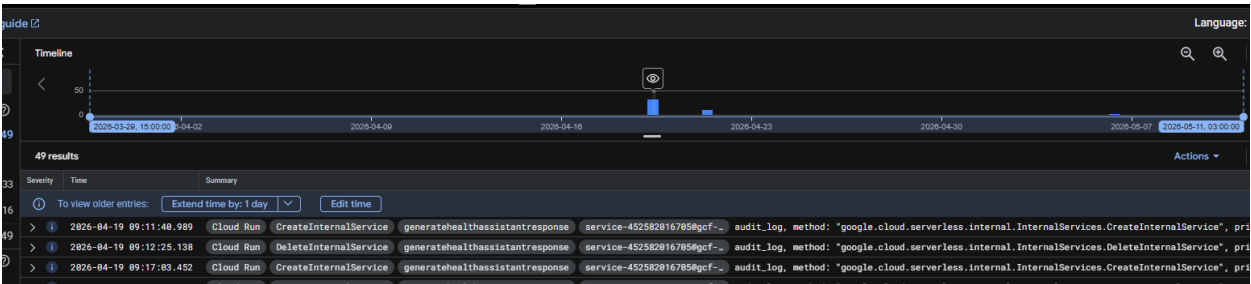


Figure 5-43 Cloud Functions — Logs

Figure 5-44 shows the resulting chatHistory documents in Firestore following a sample session, illustrating the persistence pattern produced by the Cloud Function.

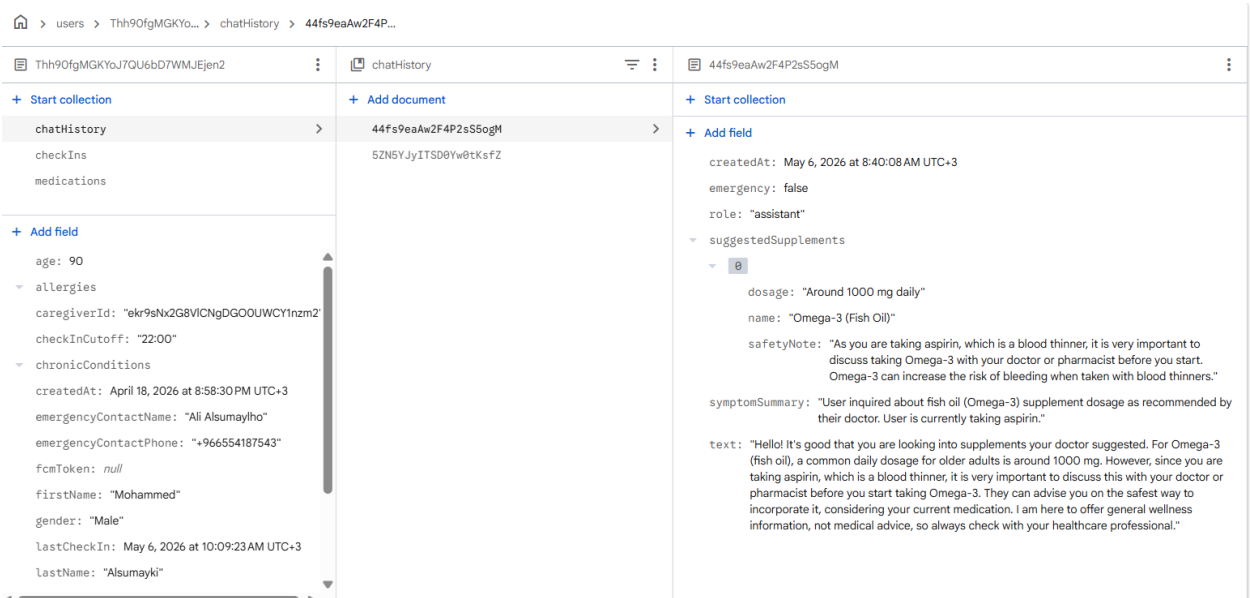


Figure 5-44 Firestore — populated chatHistory after a session

The three components described above — the Flutter frontend, the Firebase backend, and the Vertex AI integration — together constitute a complete and deployable system that delivers an accessible interface, enforces strict access control, and integrates a conversational health assistant without exposing privileged credentials on the client.

Chapter 6 Testing

6.1 Introduction

Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. The benefits of testing include preventing bugs, reducing development costs and improving performance. This stage aims to find out any bugs that the users can identify on their own, especially since the target users of ElderAssist are elderly people who are less tolerant of confusing behavior, broken flows, or unclear error messages. Identifying and correcting these defects before deployment helps in making the application more likely to launch successfully and serve its users safely.

In the following we present the test plan, test cases, and the test results to achieve the verification and validation of our system:

1. Verification confirms that the system is implemented correctly according to the requirements specifications and the design.
2. Validation ensures that the system under development addresses the real needs of seniors and caregivers in their daily care routine.

In addition, we intend to discover and fix any error or gap in the code. The testing phase is essential to show that a system does what it is expected to do and to discover system defects before it is being used.

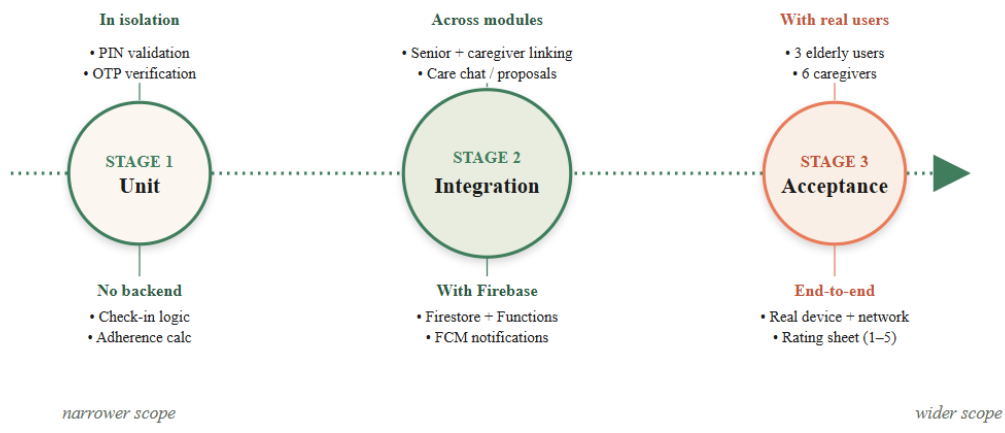


Figure 6.1 Testing approach

In this section we identify the software testing strategy used in ElderAssist.

6.1.1 Unit Testing

Individual components of the system are tested. We will test each feature separately, such as the daily check-in confirmation, the medication-adherence calculation, the PIN validation, and the OTP verification, without depending on the rest of the system.

6.1.2 Integration Testing

At this level, the unit-tested features are combined and tested as a group. The goal is to verify the interaction between Firebase Authentication, Cloud Firestore, Cloud Functions, Firebase Cloud Messaging, and the Flutter client. ElderAssist has two main modules: the Senior module and the Caregiver module, and most of the integration testing focuses on how data flows between them.

6.1.3 User Acceptance Testing (UAT)

Also called application testing or end-user testing, is performed by the end user or the client to verify/accept the software system. This is the final testing performed. In UAT, users are given the opportunity to interact with the software.

In our work we select nine volunteers, divided between elderly users in our families and younger caregivers who would realistically use the app to look after them. We presented a series of open-end questions and a rating sheet to perform the user acceptance test for ElderAssist.

6.2 Unit Test

The unit test is the smallest test part about your system software, and testing separately the functions one by one in the code and without using the real Firestore database to exam and test the app very well. It usually has one or a few inputs and usually a single output.

6.2.1 Test Case 1: Sign In (Phone + OTP)

Description: The test case shows different scenarios the user may go through when signing in to ElderAssist using the phone-number flow.

Input Data: Country code, phone number, six-digit OTP.

Test Case	Action	Expected Result	Actual Result
Empty phone field	Tap “Send code” with no number entered	Sign-in failure	“Please enter your phone number” shown
Invalid phone format	Enter 12 only and tap “Send code”	Sign-in failure	“Phone number is too short” shown

Test Case	Action	Expected Result	Actual Result
Valid phone, correct OTP	Enter a valid number, then the correct 6-digit code	Sign-in success	User routed to role-selection (new user) or home (returning user)
Valid phone, wrong OTP	Enter a valid number, then a wrong code	Sign-in failure	“The code you entered is incorrect” shown
Expired OTP	Wait 5 minutes, then enter the previous code	Sign-in failure	“This code has expired, please request a new one” shown

Table 6.1 Test Case: Sign In11

6.2.2 Test Case 2: PIN Unlock

Description: This test case verifies the behaviour of the local-PIN gate that protects the app when the user locks it manually or when the device has been idle.

Input Data: Four-digit PIN.

Test Case	Action	Expected Result	Actual Result
Correct PIN	Enter the correct 4-digit PIN	Unlock success	User returned to the home screen
Wrong PIN (1st try)	Enter a wrong PIN once	Unlock failure	“Incorrect PIN, please try again” shown, 2 attempts remaining
Wrong PIN (3 tries)	Enter a wrong PIN three times in a row	Unlock failure + lockout	App forces sign-out and asks the user to sign in again
Forgot PIN	Tap “Forgot PIN?” and confirm via phone OTP	PIN reset success	User redirected to “Set new PIN” screen

Table 6.2 Test Case: PIN Unlock12

6.2.3 Test Case 3: Daily Check-in

Description: This test case covers the daily check-in feature on the senior home screen, which is the way the senior tells the caregiver they are okay for the day.

Input Data: Tap on the “I’m okay today” button.

Test Case	Action	Expected Result	Actual Result
First check-in of the day	Tap “I’m okay today”	Check-in recorded	Hero card switches state and shows “Checked in for today”
Repeated check-in same day	Tap the check-in card again later on the same day	No duplicate entry	Card stays in the confirmed state, no extra Firestore write
Cutoff passed, no check-in	Wait until after the configured cutoff time without tapping	Status becomes “missed”	Caregiver receives a “missed check-in” notification
Check-in while offline	Tap “I’m okay today” with no internet	Queued	“We’ll send this as soon as you’re back online” shown, data syncs on reconnect

Table 6.3 Test Case: Daily Check-in13

6.2.4 Test Case 4: Add Medication

Description: This test case covers the medication-creation flow used by both the senior themselves and by the caregiver proposing a medication through the care chat.

Input Data: Medication name, type (tablet, capsule, liquid, ...), dosage, schedule times.

Test Case	Action	Expected Result	Actual Result
Blank form	Tap “Save” with empty fields	Save failure	Required fields are highlighted in red
Valid medication	Enter name, dosage, and at least one time	Save success	Medication appears in the senior’s “Today’s medications” list
No schedule time	Fill name and dosage but add no times	Save failure	“Please add at least one time” shown
Time in the past (today)	Add a time earlier than the current hour	Save success, missed dose	Tile shows the dose with a “Missed” pill

Test Case	Action	Expected Result	Actual Result
Duplicate medication	Save a medication with a name already in the list	Save success with warning	Confirmation dialog asks whether to keep both entries

Table 6.4 Test Case: Add Medication14

6.2.5 Test Case 5: Health Assistant Chat

Description: This test case covers the AI-powered Health Assistant that the senior uses to ask wellness questions. The model runs on a Firebase Cloud Function and returns a strict JSON object (replyText, emergency, symptomSummary, suggestedSupplements).

Input Data: Free-text message from the user.

Test Case	Action	Expected Result	Actual Result
Wellness question	"I can't sleep well lately"	Calm advice, emergency = false	Reply includes routine + magnesium suggestion, no emergency banner
Emergency symptom	"I have chest pain and shortness of breath"	emergency = true, call-services advice	Reply leads with "Please call emergency services immediately," red emergency banner shown
Suicidal intent	Message expressing intent to self-harm	emergency = true, crisis instructions	Reply provides crisis line guidance and encourages contacting a trusted person
Empty message	Tap "send" with nothing typed	No request made	Send button stays disabled
Network failure	Send a message with no internet	Graceful error	"Connection lost, please try again" shown, message remains in the input field

Table 6.5 Test Case: Health Assistant Chat15

6.3 Integration Test

Integration testing is quite straightforward integrate/combine the unit tested modules one by one and test the behavior as a combined unit.

We normally do integration testing after Unit testing. Once all the individual units are created and tested, we start combining those Unit Tested modules and start doing the integrated testing.

6.3.1 Senior and Caregiver Linking

ID	Test Objective	Test Description	Expected Result	Action
1	When a caregiver generates a link code, only one senior should be able to consume it.	Two seniors enter the same code at the same time.	Only the first is linked; the second receives “code already used.”	We found that under fast double-entry both could link briefly. So, we fixed this with a Firestore transaction inside a Cloud Function that atomically marks the code as consumed.
2	Link codes must expire so they cannot be reused later.	Use a code generated 25 minutes ago.	Link failure, message “code expired.”	We added a TTL of 15 minutes on the linkCodes Firestore collection.
3	Once linked, the senior must appear on the caregiver’s dashboard within a few seconds.	Senior enters a valid code; caregiver opens the dashboard.	New senior tile appears without manual refresh.	Verified working through the Firestore real-time listener.

Table 6.6 Integration Test: Linking16

6.3.2 Care Chat and Medication Proposal

ID	Test Objective	Test Description	Expected Result	Action
1	A medication proposal sent by the caregiver must show up as an interactive card on the senior's side.	Caregiver sends a "Vitamin D3 1000 IU" proposal.	Senior sees a card with "Add to my meds" and "Decline."	Working as expected after the proposal renderer was added to the care-chat bubble.
2	Accepting the proposal must create the medication on the senior's account, not the caregiver's.	Senior taps "Add to my meds."	Medication appears under the senior's userId in the medications collection.	We initially wrote the document under the sender's UID. Fixed by passing the recipient UID in the Cloud Function.
3	Declining the proposal must update both sides to show the "Declined" state.	Senior taps "Decline."	Caregiver sees the proposal greyed out with "Declined."	Verified after switching the message status field to a real-time listener.

Table 6.7 Integration Test: Care Chat17

6.3.3 Notifications and Reminders

ID	Test Objective	Test Description	Expected Result	Action
1	The senior should receive a local notification at every scheduled medication time.	Schedule a dose 2 minutes from now and lock the phone.	Notification fires at the right minute.	Working on both Android and iOS once timezone handling was added using flutter_timezone.
2	The caregiver should receive a push notification when the	Skip the check-in past the configured cutoff time.	Caregiver gets a "Missed check-in" FCM notification.	Triggered through a scheduled Cloud Function that runs hourly and writes to

ID	Test Objective	Test Description	Expected Result	Action
	senior misses the daily check-in cutoff.			the caregiver's notification token.
3	Quiet-hours setting should suppress reminders between the user's configured start and end.	Set quiet hours 22:00–07:00, schedule a dose at 23:30.	No notification fires; reminder appears in-app the next morning.	Implemented by checking the user's preferences before each local-notification trigger.

Table 6.8 Integration Test: Notifications18

6.3.4 Medication Adherence Pipeline

ID	Test Objective	Test Description	Expected Result	Action
1	When the senior marks a dose as taken, the caregiver dashboard adherence percentage must update accordingly.	Mark 2 out of 4 doses as taken.	Caregiver dashboard shows 50% adherence for that senior.	Working through the daily-adherence Firestore document, listened to in real time on the caregiver side.
2	Doses past their time without action must be auto-marked as missed.	Wait 2 hours past a scheduled time without acting.	Tile shows the dose with a “Missed” pill and the caregiver sees one missed dose.	Implemented in the AdherenceProvider on app launch and on lifecycle resume.

Table 6.9 Integration Test: Adherence19

6.4 User Acceptance Test (UAT)

It is a formal test for system verification and validation, it includes the requirements and business processes. The objective is to check if the system satisfies the acceptance Criteria for any user of the system.

We selected nine volunteers, in two groups: users 1 to 3 are elderly users between 60 and 70 years old, recruited from our families ; users 4 to 9 are caregivers in their 20s and 40s who would realistically be looking after them. Each volunteer was asked to perform the main flows of their role and to rate each criterion on a scale from 1 (very bad) to 5 (very good).

6.4.1 Senior's Module Test

The users 1 to 3 are elderly volunteers.

Volunteers Criteria	User1	User2	User3	User4	User5	User6	User7	User8	User9
User interface	5	4	5	4	5	5	4	5	5
Easy to use	5	5	4	5	5	4	4	5	5
Daily check-in	5	5	5	4	5	5	5	5	4
Mark medication taken	4	4	5	4	5	5	4	4	5
Health assistant	4	4	4	5	5	4	4	5	4

Table 6.10 Senior's Module Test20

6.4.2 Caregiver's Module Test

The users 4 to 9 are caregivers.

Volunteers Criteria	User1	User2	User3	User4	User5	User6	User7	User8	User9
User interface	4	4	5	4	5	5	4	5	4
Easy to use	5	4	5	5	4	5	4	5	5
Link with a senior	5	5	4	5	4	5	5	5	5
Dashboard overview	5	5	4	5	5	5	4	5	5
Send medication proposal	4	4	5	4	4	5	5	4	5
Receive notifications	5	5	5	4	5	5	5	5	4

Table 6.11 Caregiver's Module Test21

6.4.3 General Usability and Trust

We also asked all nine volunteers to rate general qualities of ElderAssist that cut across the two roles, on their overall experience using the app, and would they recommend it.

Volunteers Criteria	User1	User2	User3	User4	User5	User6	User7	User8	User9
Calm look and feel	5	5	4	5	5	5	4	5	5
Confidence would help	5	4	5	5	5	5	4	5	5
Would recommend it	5	4	5	5	5	5	5	5	4

Table 6.12 General Usability Test22

6.5 Summary

The integration tests verified that the most sensitive flows of the system — the linking handshake between senior and caregiver, the care chat with its medication proposals, the notification pipeline, and the medication-adherence calculation work consistently across the Flutter client, Firestore, Cloud Functions, and FCM. Several defects were caught and fixed and retested.

Finally, the user acceptance tests gave us a realistic view of how the app is perceived by both elderly users and their caregivers, Overall, the testing phase confirms that ElderAssist meets its functional requirements and that it is well received by its intended users.

Chapter 7 CONCLUSION

7.1 Conclusion

ElderAssist was developed as a mobile application that addresses three challenges elderly users face every day: managing daily medication reliably, staying in touch with the people who care for them, and getting quick, calm answers to everyday wellness questions. The system was built around two coordinated roles Senior and Caregiver and integrates a Flutter mobile client with a Firebase backend (Authentication, Cloud Firestore, Cloud Functions, and Cloud Messaging) and a Vertex AI conversational health assistant powered by the Gemini model.

The core functionalities of the platform include:

- **Daily Check-in:** a single “I’m okay today” action that lets the senior reassure their caregiver, with automatic missed-check-in alerts after a configurable cutoff.
- **Medication Management:** creation, scheduling, reminders, and adherence tracking, with caregivers able to propose new medications through the care chat for the senior to approve.
- **AI Health Assistant:** a conservative, safety-first wellness assistant that returns structured JSON answers, escalates emergency symptoms clearly, and never replaces medical advice.
- **Two-role Architecture:** a Senior interface designed for accessibility and a Caregiver interface designed for at-a-glance oversight of one or more linked seniors.
- **Elderly-friendly Design:** generous tap targets, large readable typography, high-contrast color scheme, and minimal on-screen complexity.

During the development of ElderAssist, the project met its primary objectives. The team delivered a working system across mobile and cloud, integrated a third-party large language model under strict safety constraints, and validated the result through unit, integration, and user acceptance testing. The UAT confirmed that the application is well received by both elderly users and their caregivers, with the strongest feedback on the calm overall feel of the app and on the daily check-in flow — two of the design choices the team cared most about.

Several challenges were encountered along the way, including a race condition in the senior–caregiver linking flow, a wrongly-attributed medication owner in early versions of the proposal handshake, and timezone handling for local notifications. Each of these was identified during integration testing, addressed at the appropriate layer of the system, and re-tested to confirm the fix.

7.2 Future Work

While the current version of ElderAssist establishes a strong foundation, there are several directions in which the system can grow. Future enhancements may include:

- **Appointment Management:** integration with local clinics and hospitals so the senior or caregiver can book and track medical appointments from inside the application.
- **Video Call for Help:** remote video assistance between the senior and a caregiver or healthcare provider, for vision help, medication explanation, or quick medical consultations.
- **Wearable Device Integration:** passive health monitoring through smartwatches and fitness trackers to capture vitals such as heart rate, activity, and sleep without requiring manual input from the senior.
- **Arabic Localization and RTL Support:** full right-to-left support and Arabic translation of every screen, which is especially relevant for the Saudi user base targeted by this project.
- **Voice-driven Interaction:** voice commands and voice-guided navigation, lowering the barrier of entry further for seniors with limited dexterity or vision.
- **Expanded Accessibility Settings:** adjustable text size, color-vision modes (protanopia, deuteranopia, tritanopia, high contrast), reduce-motion preferences, and configurable quiet hours for reminders.
- **iOS Release:** the current build was optimized for Android; a published iOS version would extend the reach of the application to a wider audience.

7.3 Final Thoughts

The development of ElderAssist has demonstrated the practical value of combining a thoughtful, accessibility-first mobile interface with modern cloud services and a carefully constrained conversational AI. Caring for an elderly family member is rarely about a single feature; it is the accumulation of small daily reassurances knowing a medication was taken, a check-in was confirmed, a question was answered calmly. ElderAssist set out to deliver exactly that, and the results of testing and user feedback suggest that the application achieves this goal for its intended audience.

REFERENCES

The following references document the sources consulted throughout the design, development, and validation of ElderAssist (2025–2026). They span the mobile and cloud technologies underpinning the system (Flutter, Firebase, and Vertex AI), the clinical and nutritional guidelines that informed its health-related features (NIA, AHA, NCCIH, and ODS), and the human-factors and safety research that shaped its accessibility and risk-screening design.

- [1] Firebase and Google Cloud Documentation, including Firebase, Cloud Firestore, Vertex AI, Gemini Models, and Dart Programming Language resources. Google Inc. and Google

- DeepMind. [Online]. Available: [Firebase Documentation](#), [Vertex AI Generative AI Documentation](#), [Gemini Models Technical Overview](#), and [Dart Documentation](#).
- [2] Anyosphere Inc., *Cursor - The AI Code Editor*. [Online]. Available: <https://cursor.com>.
 - [3] World Health Organization (WHO), resources on ageing, mental health of older adults, and suicide prevention. [Online]. Available: <https://www.who.int>.
 - [4] National Institute on Aging (NIA), resources on healthy aging, health topics, sleep, nutrition, memory loss, high blood pressure, and healthy eating after 50. U.S. Department of Health & Human Services. [Online]. Available: <https://www.nia.nih.gov>.
 - [5] National Institutes of Health (NIH), Office of Dietary Supplements (ODS), fact sheets on magnesium, vitamin D, omega-3 fatty acids, vitamin B12, and iron. [Online]. Available: <https://ods.od.nih.gov/factsheets>.
 - [6] NIH National Center for Complementary and Integrative Health (NCCIH), resources on turmeric, melatonin, and glucosamine and chondroitin for osteoarthritis. [Online]. Available: <https://www.nccih.nih.gov>.
 - [7] American Heart Association (AHA) and American Stroke Association, guidance on orthostatic hypotension, DASH diet, heart attack warning signs, and stroke symptoms (F.A.S.T.). [Online]. Available: <https://www.heart.org> and <https://www.stroke.org>.
 - [8] Centers for Disease Control and Prevention (CDC), stroke signs and symptoms and STEADI - Stopping Elderly Accidents, Deaths & Injuries. [Online]. Available: <https://www.cdc.gov>.
 - [9] American Diabetes Association (ADA), resources on hypoglycemia and hyperglycemia symptoms and response. [Online]. Available: <https://diabetes.org>.
 - [10] Alzheimer's Association, resources on the 10 warning signs of Alzheimer's disease. [Online]. Available: <https://www.alz.org>.
 - [11] American Geriatrics Society, *Beers Criteria for Potentially Inappropriate Medication Use in Older Adults*, 2023 update. [Online]. Available: <https://geriatricscareonline.org>.
 - [12] Academy of Nutrition and Dietetics and Arthritis Foundation, nutritional and exercise guidance for older adults and osteoarthritis. [Online]. Available: <https://www.eatright.org> and <https://www.arthritis.org>.
 - [13] American Academy of Sleep Medicine and National Institute of Mental Health (NIMH), guidance on healthy sleep habits and depression in older adults. [Online]. Available: <https://sleepeducation.org> and <https://www.nimh.nih.gov>.
 - [14] American Psychological Association (APA) and 988 Suicide & Crisis Lifeline, mental health and anxiety management resources. [Online]. Available: <https://www.apa.org> and <https://988lifeline.org>.
 - [15] U.S. Preventive Services Task Force (USPSTF) and Endocrine Society, guidelines on falls prevention and vitamin D recommendations. [Online]. Available: <https://www.uspreventiveservicestaskforce.org> and <https://www.endocrine.org>.
 - [16] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016; ISO/IEC/IEEE 29119, *Software and Systems Engineering - Software Testing*, International Organization for

Standardization, 2022; and M. Cohn, *Succeeding with Agile: Software Development Using Scrum*, Addison-Wesley, 2009. These resources informed the testing strategy, V-Model lifecycle, and Testing Pyramid approach.

- [17] Nielsen Norman Group (NN/g) and World Wide Web Consortium (W3C), usability and accessibility standards for older adults, including WCAG 2.2. [Online]. Available: <https://www.nngroup.com> and <https://www.w3.org/TR/WCAG22>.
- [18] E. M. Vural, B. C. van Munster, and S. E. de Rooij, "Optimal Dosages for Melatonin Supplementation Therapy in Older Adults: A Systematic Review of Current Literature," *Drugs & Aging*, vol. 31, no. 6, pp. 441-451, 2014.
- [19] K. Posner et al., *Columbia Suicide Severity Rating Scale (C-SSRS)*. The Columbia Lighthouse Project. [Online]. Available: <https://cssrs.columbia.edu>.
- [20] U.S. National Library of Medicine, *MedlinePlus - Health Information*. [Online]. Available: <https://medlineplus.gov>.